

智能体上下文工程：为自我改进的语言模型演化上下文

Qizheng Zhang^{1*}, Changran Hu^{2*}, Shubhangi Upasani², Boyuan Ma², Fenglu Hong²,
Vamsidhar Kamanuru², Jay Rainton², Chen Wu², Mengmeng Ji², Hanchen Li³,
Urmish Thakker², James Zou¹, Kunle Olukotun¹

¹Stanford University ²SambaNova Systems, Inc. ³UC Berkeley

{qizhengz, kunle}@stanford.edu changran_hu@berkeley.edu

🔗 ace-agent/ace 🌐 ace-agent.github.io *Equal contribution.

摘要

大语言模型（LLM）应用，如智能体和领域特定推理，越来越依赖于上下文适应：通过修改输入中的指令、策略或证据，而非更新权重。先前的方法提升了可用性，但往往受到简洁性偏差的影响，即为了简洁摘要而丢失领域洞见；也受到上下文坍塌的影响，即迭代重写会逐渐侵蚀细节。我们提出 **ACE**（智能体上下文工程），一个将上下文视为不断演化的策略手册的框架，通过生成、反思和策展的模块化过程来积累、精炼和组织策略。**ACE** 通过结构化的增量更新防止坍塌，保留详细知识并适应长上下文模型。在智能体和领域特定基准上，**ACE** 在离线（例如系统提示）和在线（例如智能体记忆）场景中优化上下文，始终优于强基线：智能体任务上提升 +10.6%，金融任务上提升 +8.6%，同时显著降低适应延迟和部署成本。值得注意的是，**ACE** 无需标注监督即可有效适应，而是利用自然执行反馈。在 **AppWorld** 排行榜上，**ACE** 在总体平均分上与排名第一的生产级智能体持平，并在更难的测试挑战集上超越它，尽管使用的是更小的开源模型。这些结果表明，全面且不断演化的上下文能够以低开销实现可扩展、高效且自我改进的 LLM 系统。

1 引言

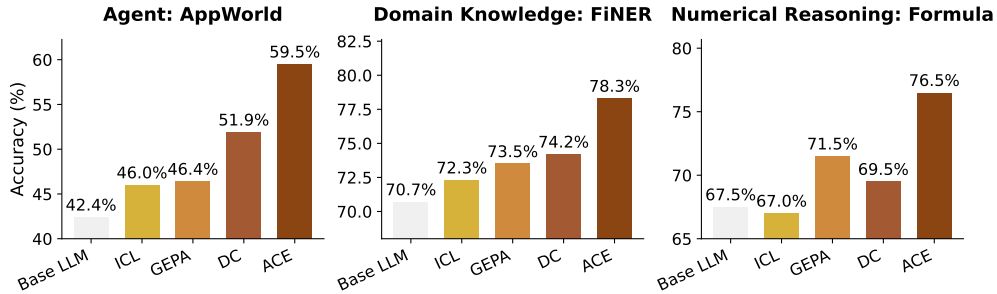


图 1：总体性能结果。我们提出的框架 ACE 在智能体和领域特定任务上始终优于强基线。

基于大语言模型（LLM）的现代 AI 应用，如 LLM 智能体（Yao 等，2023；Yang 等，2024）和复合 AI 系统（Zaharia 等，2024），越来越依赖于上下文适应。上下文适应不修改模型权重，而是在模型训练后通过引入清晰的指令、结构化的推理步骤或领域知识来提升性能。

将特定输入格式直接注入模型的输入中。上下文支撑着众多人工智能系统组件，包括引导下游任务的系统提示（Opsahl-Ong 等人，2024；Agrawal 等人，2025）、承载过往事实与经验的记忆（Suzgun 等人，2025；Xu 等人，2025），以及减少幻觉并补充知识的事实证据（Asai 等人，2024）。

通过上下文而非权重进行适应具有若干关键优势。上下文对用户和开发者而言是可解释且可说明的（Wei 等人，2022；Wang 等人，2023），允许在运行时快速整合新知识（Lewis 等人，2020；Borgeaud 等人，2022），并且可以在复合系统中的不同模型或模块之间共享（Khot 等人，2023）。与此同时，长上下文大语言模型（Peng 等人，2024）以及诸如键值缓存复用等上下文高效推理技术（Gim 等人，2024；Yao 等人，2025）的进步，正使得基于上下文的方法在部署中日益实用。因此，上下文适应正逐渐成为构建功能强大、可扩展且能自我改进的人工智能系统的核心范式。

尽管取得了这些进展，现有的上下文适应方法仍面临两个局限性。第一，简洁性偏差：许多提示优化器优先考虑简洁适用的指令，而非全面积累。例如，GEPA（Agrawal 等人，2025）将简洁性视为优势，但此类抽象可能遗漏领域特定的启发式规则、工具使用指南或实践中常见的失败模式（Gao 等人，2025）。该目标在某些场景下与验证指标一致，但往往无法捕捉智能体和知识密集型应用所需的详细策略。第二，上下文坍塌：依赖大语言模型进行整体重写的方法往往会随时间退化为更短、信息量更少的摘要，导致性能急剧下降（图 2）。在交互式智能体（Trivedi 等人，2024；Patil 等人，2024；Zhang 等人，2024）、领域特定编程（Ye 等人，2023；Zhang 等人，2025a；b；Mang 等人，2025）以及金融或法律分析（Loukas 等人，2022；Guha 等人，2023；Wang 等人，2025a）等领域，强大的性能依赖于保留详细的任务特定知识，而非将其压缩掉。

随着智能体与知识密集型推理等应用对可靠性提出更高要求，近期研究转向利用长上下文大语言模型（Long-Context LLMs）的进展，将上下文填充为丰富且潜在有用的信息（Jiang et al., 2025; Chung et al., 2025; Chen et al., 2025）（Peng et al., 2024; Mao et al., 2024）。本文认为，上下文不应作为简洁摘要，而应作为全面、结构化的操作手册，详尽、包容且富含领域知识。

洞见。与人类常受益于简洁概括不同，大语言模型在获得冗长且详细的上下文时更为有效，并能自主提炼相关信息（Jiang 等，2025；Liu 等，2025b；Suzgun 等，2025）。因此，不应压缩掉领域特定的启发式方法与策略，而应在上下文中保留它们，使模型在推理时自行判断何为关键。

为解决这些局限性，本文提出智能体上下文工程（**Agentic Context Engineering, ACE**）框架，用于在离线场景（如系统提示优化）和在线场景（如测试时记忆自适应）中实现全面的上下文自适应。**ACE** 不将上下文压缩为提炼摘要，而是将其视为不断演化的操作手册，随时间积累并组织策略。通过设计，**ACE** 融合了生成、反思与策展的模块化工作流，同时依据增长与精炼原则进行结构化增量更新。该设计保留了详细的领域特定知识，防止上下文坍塌，并产生在自适应过程中始终保持全面且可扩展的上下文。

我们在两类最能受益于全面且不断演进的上下文的大语言模型（LLM）应用上评估了 ACE：（1）智能体（Trivedi et al., 2024），这类应用需要多轮推理、工具使用和环境交互，其中积累的策略可以在不同回合间复用；（2）领域特定基准，这类基准要求专门的策略和知识，例如金融分析（Loukas et al., 2022; Wang et al., 2025a）。我们的主要发现如下：

- ACE 始终优于强基线，在智能体和领域特定基准上分别平均提升 10.6% 和 8.6%，涵盖离线和在线适应设置。
- ACE 能够在无标签监督的情况下构建有效上下文，而是利用执行反馈和环境信号，这是自改进大语言模型和智能体的关键要素。
- 在 AppWorld 基准排行榜（AppWorld）上，ACE 超越了排名第一的生产级智能体 IBM-CUGA（Marreed 等，2025）（由 GPT-4.1 驱动），同时使用开源模型（DeepSeek-V3.1）。

- ACE 所需的 rollout 次数显著减少，且适应延迟低于现有自适应方法，表明可扩展的自改进能够以更高的准确率和更低的成本实现。

2 背景与动机

2.1 上下文适应

上下文适应（或上下文工程）指通过构建或修改大语言模型的输入而非改变其权重来改进模型行为的方法。当前最新技术利用自然语言反馈（Shinn 等，2023；Yuksekgonul 等，2025；Agrawal 等，2025）。在该范式中，语言模型检查当前上下文以及执行迹、推理步骤或验证结果等信号，并生成关于如何修订上下文的自然语言反馈。该反馈随后被纳入上下文，实现迭代适应。代表性方法包括 Reflexion（Shinn 等，2023），它反思失败以改进智能体规划；TextGrad（Yuksekgonul 等，2025），它通过类似梯度的文本反馈优化提示；GEPA（Agrawal 等，2025），它基于执行迹迭代精化提示，并在某些设置中取得强大性能，甚至超越强化学习方法；以及 Dynamic Cheatsheet（Krause 等，2019），它构建外部记忆，在推理过程中积累过去成功与失败的策略和教训。这些自然语言反馈方法代表了一项重大进展，为超越权重更新改进大语言模型系统提供了灵活且可解释的信号。

2.2 现有上下文适应方法的局限性

简洁性偏差 上下文自适应方法的一个反复出现的局限性是简洁性偏差：优化倾向于坍塌为简短、通用的提示。Gao 等人（Gao et al., 2025）在测试生成的提示优化中记录了这种效应，其中迭代方法反复产生几乎相同的指令（例如，“创建单元测试以确保方法按预期行为”），牺牲了多样性并省略了领域特定的细节。这种收敛不仅缩小了搜索空间，而且还在迭代过程中传播反复出现的错误，因为优化后的提示往往继承与其种子相同的缺陷。更广泛地说，这种偏差削弱了在需要详细、上下文丰富指导的领域中的性能——例如多步智能体、程序合成或知识密集型推理——在这些领域中，成功取决于积累而非压缩任务特定的洞见。

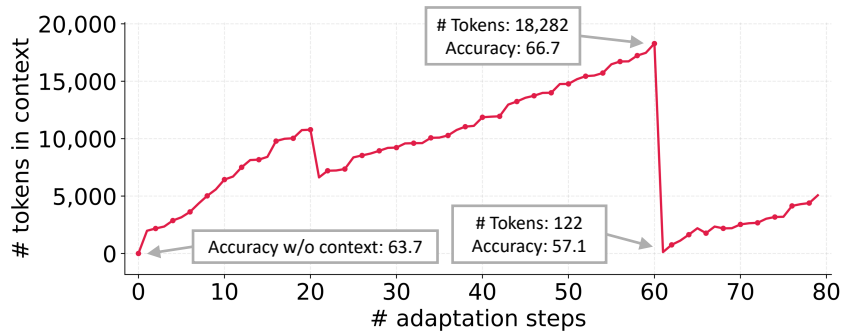


图 2：上下文坍塌。由大语言模型对上下文进行整体重写可能将其坍塌为更短、信息量更少的摘要，导致性能急剧下降。

上下文坍塌 在 AppWorld 基准（Trivedi et al., 2024）的案例研究中，我们观察到一种我们称之为上下文坍塌的现象，这种现象出现在让大语言模型在每个自适应步骤完全重写累积的上下文时。随着上下文变得庞大，模型倾向于将其压缩为更短、信息量更少的摘要，导致信息的大量丢失。例如，在第 60 步，上下文包含 18,282 个标记，准确率达到 66.7，但在紧接着的下一步，它坍塌到仅 122 个标记，准确率下降到 57.1——比没有自适应的基线准确率 63.7 还要差。虽然我们通过 Dynamic Cheat-

sheet (Suzgun et al., 2025) 强调了这一点，但问题并非该方法所特有；相反，它反映了使用大语言模型进行端到端上下文重写的一个根本风险，即累积的知识可能被突然抹去而不是被保留。

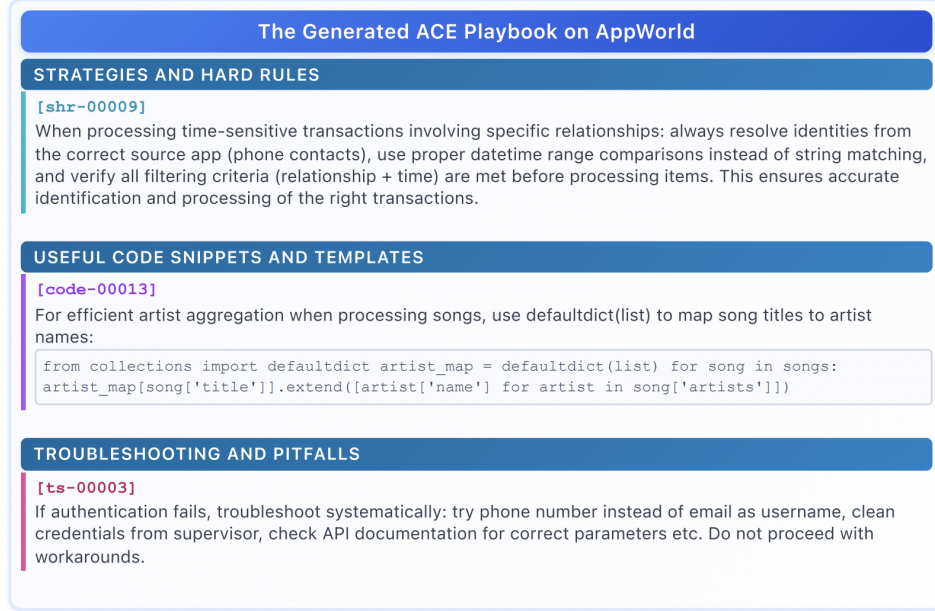


图 3: AppWorld 基准上 ACE 生成上下文的示例（部分展示）。ACE 生成的上下文包含详细的、领域特定的洞见以及随时可用的工具和代码，可作为大语言模型应用的全面操作手册。

3 智能体上下文工程 (ACE)

我们提出 **ACE**（智能体上下文工程，**Agentic Context Engineering**），一个用于在离线（如系统提示优化）和在线（如测试时记忆自适应）场景中实现可扩展且高效上下文自适应的框架。**ACE** 不将知识压缩为简短的摘要或静态指令，而是将上下文视为不断演化的操作手册，随时间持续积累、精炼和组织策略。受动态速查表（**Dynamic Cheatsheet**, Suzgun 等, 2025）的智能体设计启发，**ACE** 引入了三个角色之间的结构化分工（图 4）：生成器（**Generator**）产生推理轨迹；反思器（**Reflector**）从成功和错误中提炼具体洞见；策展器（**Curator**）将这些洞见整合为结构化的上下文更新。这反映了人类学习的方式：实验、反思和巩固，同时避免了将全部责任加载到单一模型上造成的瓶颈。

为了解决 § 2.2 中讨论的先前方法的局限性（尤其是简洁性偏差和上下文坍塌），ACE 引入了三项关键创新：（1）专用的反思器，将评估和洞见提取与策展分离，提高上下文质量和下游性能（§ 4.6）；（2）增量式增量更新（§ 3.1），用局部编辑取代昂贵的整体重写，降低延迟和计算成本（§ 4.7）；（3）增长与精炼机制（§ 3.2），在稳定的上下文扩展与冗余控制之间取得平衡。

如图 4 所示，工作流程始于生成器为新查询产生推理轨迹，这些轨迹既揭示了有效策略，也暴露了反复出现的陷阱。反思器对这些轨迹进行批判，提取经验教训，并可选地在多次迭代中加以精炼。策展器随后将这些经验教训综合为紧凑的增量条目，由轻量级、非大语言模型（LLM）逻辑确定性地合并到现有上下文中。由于更新是逐项且局部化的，多个增量可以并行合并，从而支持大规模批量自适应。ACE 进一步支持多训练轮次自适应，即重新访问相同查询以逐步强化上下文。

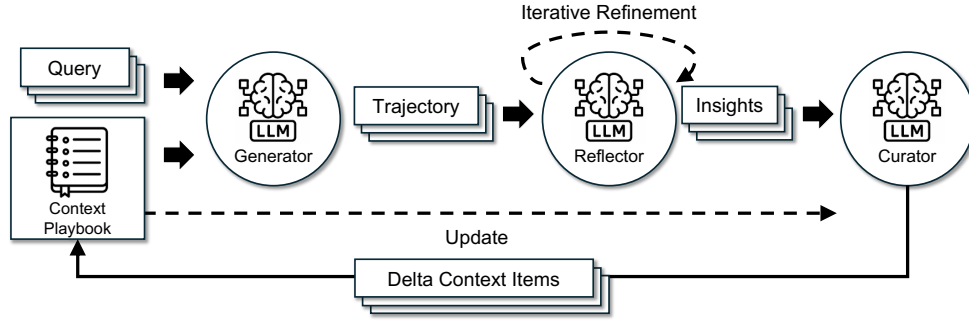


图 4: ACE 框架。受动态速查表启发，ACE 采用智能体架构，包含三个专门组件：生成器、反思器和策展器。

3.1 增量式增量更新

ACE 的一个核心设计原则是将上下文表示为一系列结构化、逐条列出的要点集合，而非单一的庞大提示。要点的概念类似于动态备忘单（Dynamic Cheatsheet）（Suzgun 等，2025）和 A-MEM（Xu 等，2025）等大语言模型记忆框架中的记忆条目概念，但在此基础上进行了扩展，包含（1）元数据，包括唯一标识符以及记录其被标记为有帮助或有危害次数的计数器；以及（2）内容，捕捉一个小的单元，例如可复用的策略、领域概念或常见失败模式。在解决新问题时，生成器会突出显示哪些要点有用或具有误导性，从而提供反馈以指导反思器提出修正性更新。

这种逐条设计实现了三个特性：（1）局部化，仅更新相关的要点；（2）细粒度检索，使生成器能够聚焦于最相关的知识；（3）增量适应，允许在推理过程中高效地合并、剪枝和去重。

ACE 并非完全重新生成上下文，而是增量地产生紧凑的增量上下文：由反思器提炼并由策展器整合的小型候选要点集合。这避免了完全重写的计算成本和延迟，同时确保过去的知识得以保留，新的洞见得以持续追加。随着上下文的增长，这种方法为长期或领域密集型应用提供了所需的可扩展性。

3.2 增长与精炼

除了增量增长之外，ACE 通过定期或惰性精炼确保上下文保持紧凑和相关。在增长与精炼中，带有新标识符的要点被追加，而现有要点则被就地更新（例如，递增计数器）。随后，去重步骤通过语义嵌入比较要点来剪除冗余。这种精炼可以主动执行（在每次增量之后）或惰性执行（仅在上下文窗口超出时），具体取决于应用对延迟和准确率的要求。

增量更新与增长与精炼相结合，维持了能够自适应扩展、保持可解释性并避免由整体上下文重写引入的潜在方差的上下文。

4 结果

我们对 ACE 的评估表明：

- **实现高性能、自我改进的智能体。**ACE 通过在离线和在线设置中动态优化其输入上下文，使智能体能够自我改进。仅通过执行反馈学习构建更优上下文，无需真实标签，在 AppWorld 基准上准确率提升高达 17.1%。（§ 4.3）
- **在领域特定基准上取得显著提升。**在复杂的金融推理基准上，ACE 通过构建包含领域特定概念和洞见的综合手册，相较于强基线平均性能提升 8.6%。（§ 4.4）

- **设计有效性。**消融研究证实了我们的设计选择是成功的关键，反射器、多轮次优化和增量更新等组件均贡献了显著的性能提升。（§ 4.6）

- **更低的成本和适应延迟。**ACE 高效地实现了这些提升，平均将适应延迟降低了 86.9%，同时需要更少的执行次数和更低的令牌美元成本。（§ 4.7）

4.1 任务与数据集

我们在两类最能从不断演化的上下文中受益的大语言模型应用上评估 ACE：

（1）大语言模型智能体，需要多轮推理、工具使用和环境交互；借助 ACE，智能体可以跨回合和环境积累并复用策略；以及（2）领域特定推理，要求掌握专门的概念和策略；我们以金融分析作为主要案例研究，并展示在医学推理和文本到结构化查询语言上的额外结果。

- **大语言模型智能体：应用世界（AppWorld）**（特里维迪等人，2024）是一套自主智能体任务，涉及应用程序编程接口理解、代码生成和环境交互。它提供了一个真实的执行环境，包含常见应用和应用程序编程接口（如电子邮件、文件系统），以及两个难度级别的任务（普通和挑战）。一个公开排行榜（应用世界）跟踪性能，在提交时，最佳系统仅达到 60.3% 的平均准确率，凸显了该基准的难度和真实性。
- **领域特定推理：金融、医学和文本到结构化查询语言基准** 我们在第 4.4 节中以金融作为主要案例研究。对于金融分析，我们关注金融命名实体识别（FiNER）（卢卡斯等人，2022）和公式（Formula）（王等人，2025a），它们测试大语言模型在依赖可扩展商业报告语言（XBRL）的金融推理任务上的表现。金融命名实体识别要求用 139 种细粒度实体类型之一标注可扩展商业报告语言金融文档中的标记，这是受监管领域中金融信息提取的关键步骤。公式专注于应用金融概念并执行计算来回答查询，即数值推理。除金融外，我们还在流基准（StreamBench）（吴等人，2024）的两个额外领域任务上进行评估：鉴别诊断增强（DDXPlus）（范西·昌戈等人，2022）（医学推理）和鸟结构化查询语言（BIRD-SQL）（李等人，2023）（文本到结构化查询语言）。

评估度量 对于应用世界，我们遵循官方基准协议，并在测试普通和测试挑战划分上报告任务目标完成率（TGC）和场景目标完成率（SGC）。对于金融命名实体识别、公式和鉴别诊断增强，我们遵循原始设置并报告准确率，即预测答案与真实答案完全匹配的比例。对于鸟结构化查询语言，我们在大模型作为评审（郑等人，2023）下使用 GPT-4o-mini（开放人工智能，2024）。

所有数据集均遵循原始的训练/验证/测试划分。对于离线上上下文适应，方法在训练划分上进行优化，并在测试划分上以 pass@1 准确率进行评估。对于在线上下文适应，方法在测试划分上按顺序评估：对于每个样本，模型首先利用当前上下文进行预测，然后基于该样本更新其上下文。所有方法使用相同的打乱顺序的测试划分。

4.2 基线与方法

基础大语言模型 基础模型直接在每个基准上进行评估，不进行任何上下文工程，使用数据集作者提供的默认提示。对于 AppWorld，我们遵循基准作者发布的官方 ReAct（Yao 等人，2023）实现，并在此框架之上构建所有其他基线和方法。

上下文学习（In Context Learning (ICL)）（Agarwal 等人，2024）ICL 在输入提示中为模型提供任务演示（少样本或多样本）。这使得模型无需权重更新即可推断任务格式和期望输出。当所有训练样本都能放入模型的上下文窗口时，我们提供全部训练样本；否则，我们用尽可能多的演示填满窗口。

MIPROv2（Opsahl-Ong 等人，2024）MIPROv2 是一种流行的大语言模型应用提示优化器，通过贝叶斯优化联合优化系统指令和上下文演示。我们使用官方 DSPy 实现（DSPy，b），设置 auto="heavy" 以最大化优化性能。

GEPA (Agrawal 等人, 2025) GEPA (遗传-帕累托) 是一种基于反思式提示演化的样本高效提示优化器。它收集执行迹 (推理、工具调用、中间输出), 并应用自然语言反思来诊断错误、分配信用并提出提示更新。遗传帕累托搜索维护一个高性能提示的前沿, 以缓解局部最优。实验表明, GEPA 优于强化学习方法 (如 GRPO) 和提示优化器 (如 MIPROv2), 在减少多达 $35\times$ 次 rollout 的情况下, 准确率最高可提升 10 - 20%。我们使用官方 DSPy 实现 (DSPy, α), 并设置 `auto="heavy"` 以最大化优化性能。

动态备忘单 (DC) (Suzgun 等人, 2025) DC 是一种测试时学习方法, 引入了一个自适应外部记忆, 用于存储可复用的策略和代码片段。通过不断用新遇到的输入和输出更新该记忆, DC 使模型能够积累知识并在任务间复用, 通常比静态提示方法带来显著提升。DC 的一个关键优势是不需要真实标签: 模型可以从自己的生成中整理记忆, 使该方法高度灵活且广泛适用。我们使用作者发布的官方实现 (Suzgun 等人), 并将其设置为累积模式 (DC-CU)。

ACE (本文方法) ACE 通过智能体上下文工程框架, 针对离线和在线适应优化大语言模型上下文。为确保公平性, 我们为生成器、反思器和策展器使用相同的大语言模型 (DeepSeek-V3.1 的反思模式 (Liu 等人, 2024a)), 防止更强的反思器或策展器向较弱的生成器传递知识。这隔离了上下文构建本身的收益。我们还在附录中使用其他骨干大语言模型评估 ACE, 观察到一致的提升。我们采用批大小为 1 (从每个样本构建增量上下文)。我们将离线适应中反思器细化轮次的最大值和训练轮次的最大值设为 5。

4.3 智能体基准上的结果

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
DeepSeek-V3.1-671B as Base LLM						
ReAct		63.7	42.9	41.5	21.6	42.4
Offline Adaptation						
ReAct + ICL	✓	64.3 ^{+0.6}	46.4 ^{+3.5}	46.0 ^{+4.5}	27.3 ^{+5.7}	46.0 ^{+3.6}
ReAct + GEPA	✓	64.9 ^{+1.2}	44.6 ^{+1.7}	46.0 ^{+4.5}	30.2 ^{+8.6}	46.4 ^{+4.0}
ReAct + ACE	✓	76.2 ^{+12.5}	64.3 ^{+21.4}	57.3 ^{+15.8}	39.6 ^{+18.0}	59.4 ^{+17.0}
ReAct + ACE	✗	75.0 ^{+11.3}	64.3 ^{+21.4}	54.4 ^{+12.9}	35.2 ^{+13.6}	57.2 ^{+14.8}
Online Adaptation						
ReAct + DC (CU)	✗	65.5 ^{+1.8}	58.9 ^{+16.0}	52.3 ^{+10.8}	30.8 ^{+9.2}	51.9 ^{+9.5}
ReAct + ACE	✗	69.6 ^{+5.9}	53.6 ^{+10.7}	66.0 ^{+24.5}	48.9 ^{+27.3}	59.5 ^{+17.1}

表 1: 在 AppWorld 智能体基准上的结果 (以 DeepSeek-V3.1-671B 作为基础大语言模型)。“GT 标签”表示在适应过程中反思器是否可获得真实标签。我们在官方 ReAct 实现之上, 针对离线和在线上下文适应, 将 ACE 框架与多个基线进行了评估。ReAct + ACE 平均比所选基线高出 10.6%, 并且即使在没有 GT 标签的情况下也能取得良好性能。

分析: AppWorld 如表 1 所示, ACE 在 AppWorld 上持续优于强基线。在离线设置中, ReAct + ACE 分别以显著优势 (12.3% 和 11.9%) 超过 ReAct + ICL 和 ReAct + GEPA, 表明结构化、不断演进且详细的上下文比固定演示或单一优化指令提示更能实现有效的智能体学习。这些提升延伸到在线设置, ACE 继续以平均 7.6% 的优势超越先前的自适应方法, 如动态备忘单。

在智能体用例中, 即使适应过程中无法获得真实标签, ACE 仍然有效: ReAct + ACE 相比 ReAct 基线平均提升 14.8%

在此设置下。这种鲁棒性源于 ACE 利用执行过程中自然可用的信号（例如，代码执行成功或失败）来引导反射器和策展人形成关于成功与失败的结构化经验。综合这些结果，ACE 成为一个强大且多功能的框架，用于构建在有标签和无标签监督下都能可靠适应的自改进智能体。

值得注意的是，在最新的 AppWorld 排行榜上（截至 2025 年 9 月 20 日；图 5），ReAct + ACE（平均 59.4%）与排名第一的 IBM CUGA（60.3%）¹ 相当，后者是一个基于 GPT-4.1 的生产级智能体（Marreed 等人，2025），尽管使用的是小得多的开源模型 DeepSeek-V3.1。通过在线适应，ReAct + ACE 在测试挑战中的 TGC 和 SGC 上甚至分别超过 IBM CUGA 8.4% 和 0.7%，凸显了 ACE 在为智能体构建全面且自我演进的上下文方面的有效性。

4.4 领域特定基准上的结果

Method	GT Labels	FiNER (Acc↑)	Formula (Acc↑)	Average
DeepSeek-V3.1 as Base LLM				
Base LLM		70.7	67.5	69.1
Offline Adaptation				
ICL	✓	72.3 ^{+1.6}	67.0 ^{-0.5}	69.6 ^{+0.5}
MIPROv2	✓	72.4 ^{+1.7}	69.5 ^{+2.0}	70.9 ^{+1.8}
GEPA	✓	73.5 ^{+2.8}	71.5 ^{+4.0}	72.5 ^{+3.4}
ACE	✓	78.3^{+7.6}	85.5^{+18.0}	81.9^{+12.8}
ACE	✗	71.1 ^{+0.4}	83.0 ^{+15.5}	77.1 ^{+8.0}
Online Adaptation				
DC (CU)	✓	74.2 ^{+3.5}	69.5 ^{+2.0}	71.8 ^{+2.7}
DC (CU)	✗	68.3 ^{-2.4}	62.5 ^{-5.0}	65.4 ^{-3.7}
ACE	✓	76.7^{+6.0}	76.5 ^{+9.0}	76.6^{+7.5}
ACE	✗	67.3 ^{-3.4}	78.5^{+11.0}	72.9 ^{+3.8}

表 2：金融分析基准上的结果（以 DeepSeek-V3.1-671B 作为基础大语言模型）。“GT 标签”表示在适应过程中反射器是否可获得真实标签。在有真实标签的情况下，ACE 在离线和在线设置中均取得一致的改进，凸显了结构化且不断演化的上下文对于领域特定推理的优势。然而，我们也观察到，在缺乏可靠反馈信号（例如真实标签或执行结果）时，ACE 和其他自适应方法（如动态备忘单）都可能出现性能下降，这表明上下文适应在很大程度上依赖于反馈质量。

分析：金融基准 如表 2 所示，ACE 在金融分析基准上带来了显著的改进。在离线设置中，当提供训练划分中的真实答案时，ACE 以明显优势超越 ICL、MIPROv2 和 GEPA（平均高出 10.9%），表明当任务需要精确的领域知识（例如金融概念、XBRL 规则）且这些知识超出固定示例或单一优化提示的范围时，结构化且不断演化的上下文尤为有效。在在线设置中，ACE 继续以平均 6.2% 的优势超过先前的自适应方法（如 DC），进一步证实了智能体上下文工程在跨专业领域积累可复用洞见方面的优势。

此外，我们还观察到，当缺乏真实监督或可靠的执行信号时，ACE 和 DC 的性能都可能下降。在这种情况下，构建的上下文可能被虚假或误导性信号污染，这凸显了在没有可靠反馈的情况下进行推理时适应的潜在局限性。这表明，虽然 ACE 在丰富反馈（例如智能体任务中的代码执行结果或公式正确性）下具有鲁棒性，但其有效性取决于

¹ 我们提及 IBM CUGA 仅作为粗略的上下文参考，以表明 ACE 在 AppWorld 排行榜上处于相近的性能区间。它并未被用作方法论基线，我们也不进行直接比较。CUGA 的内部设计与 ACE 的上下文适应重点不同，所有基线均在相同设置下评估，以隔离方法学效应而非智能体工程选择。

是否具备允许反射器和策展人做出合理判断的信号。我们将在第 5 节回到这一局限性。

分析：医学与文本到 SQL 基准 虽然本小节以金融领域作为详细案例研究，但 ACE 并非仅适用于金融：我们在其他领域特定任务上也观察到一致的提升，包括医学推理和文本到 SQL，这表明相同的剧本式上下文适应方法可以跨领域迁移。完整结果见附录 § A.2。

4.5 跨大语言模型的泛化性

表 1 和表 2 使用我们的默认骨干模型（DeepSeek-V3.1），但 ACE 并不特定于该模型。我们可以在不改变算法或提示的情况下替换为其他大语言模型，并在 AppWorld 和金融基准上仍观察到一致的提升。附录 § A.1 报告了在 GPT-OSS-120B、GPT-5.1 和 Llama-3.3-70B-Instruct 上的完整结果，其中 ACE 相对于相应的基础智能体或模型均有改进。这些结果表明 ACE 是一种可跨大语言模型家族泛化的测试时上下文演化方法。

4.6 消融实验与敏感性分析

消融实验 表 3 报告了在 AppWorld 上的消融实验，分析了 ACE 的各个设计选择如何有助于有效的上下文适应。我们考察了三个因素：（1）具有迭代精炼功能的反思器，这是我们在动态备忘单之外对智能体框架的补充；

（2）多轮次适应，即在训练样本上多次精炼上下文；以及（3）离线预热，即在在线适应开始之前通过离线适应初始化上下文。此外，我们在附录 § A.5 中研究了增量上下文更新的效果，以及它为何是 ACE 性能提升的关键使能因素。

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
DeepSeek-V3.1 as Base LLM						
ReAct		63.7	42.9	41.5	21.6	42.4
Offline Adaptation						
ReAct + ACE w/o Reflector or multi-epoch	✓	70.8 ^{+7.1}	55.4 ^{+12.5}	55.9 ^{+14.4}	38.1 ^{+17.5}	55.1 ^{+12.7}
ReAct + ACE w/o multi-epoch	✓	72.0 ^{+8.3}	60.7 ^{+17.8}	54.9 ^{+13.4}	39.6 ^{+18.0}	56.8 ^{+14.4}
ReAct + ACE	✓	76.2 ^{+12.5}	64.3 ^{+21.4}	57.3 ^{+15.8}	39.6 ^{+18.0}	59.4 ^{+17.0}
Online Adaptation						
ReAct + ACE	✗	67.9 ^{+4.2}	51.8 ^{+8.9}	61.4 ^{+19.9}	43.2 ^{+21.6}	56.1 ^{+13.7}
ReAct + ACE + offline warmup	✗	69.6 ^{+5.9}	53.6 ^{+10.7}	66.0 ^{+24.5}	48.9 ^{+27.3}	59.5 ^{+17.1}

表 3: AppWorld 上的消融实验。我们研究了 ACE 的特定设计选择（迭代精炼、多轮次适应和离线预热）如何有助于高质量的上下文适应。

对反思质量的鲁棒性 ACE 对反思质量具有鲁棒性：即使使用弱得多的反思器，它仍然有效；更强的反思器只带来适度的额外收益；在嘈杂或有害的反思下，它也能优雅地退化，除了在每次迭代都进行完全对抗性更新的情况外，始终保持在基础模型之上。完整实验结果见附录 § A.4。

对超参数选择的敏感性 ACE 的增益在广泛的合理超参数设置范围内保持稳定（例如反思器细化轮数、适应轮次数以及增长与细化阈值）：性能变化适度，且 ACE 始终保持在相应基线之上。完整讨论和详细结果见附录 § A.6。

4.7 成本与速度分析

由于支持增量式“增量”上下文更新以及非大语言模型的上下文合并与去重，ACE 在降低适应成本（以 rollout 数量或令牌摄取/生成的美元成本计）和延迟方面表现出特别的优势。

例如，在 AppWorld 的离线适应中，与 GEPA 相比，ACE 实现了 82.3% 的适应延迟降低和 75.1% 的 rollout 数量减少（表 4(a)）。在

FiNER 的在线适应中，与 DC 相比，ACE 实现了 91.5% 的适应延迟降低和 83.6% 的令牌摄取/生成美元成本减少（表 4(b)）。

Method	Latency (s)↓	# Rollouts↓	Method	Latency (s)↓	Token Cost (\$)↓
ReAct + GEPA	53898	1434	DC (CU)	65104	17.7
ReAct + ACE	9517 (-82.3%)	357 (-75.1%)	ACE	5503 (-91.5%)	2.9 (-83.6%)

(a) 离线 (AppWorld)。

(b) 在线 (FiNER)。

表 4：成本与速度分析。我们测量了 ACE 相对于 GEPA（离线）和 DC（在线）的上下文适应延迟、展开次数以及美元成本。

细粒度成本分析 我们对 ACE 和 GEPA（作为代表性基线）进行了细粒度成本分析。在 AppWorld 上，ACE 在离线适应阶段显著更便宜，与 GEPA 相比，输入/输出令牌使用量减少了 **80.8%/83.6%**：ACE 避免了 GEPA 的提示验证循环，并用局部增量更新取代了重复的完整重写。在评估时，虽然 ACE 可能因更丰富的剧本而使用更多原始输入令牌，但这并不一定转化为更高的计费服务成本，因为大部分上下文通过 KV 缓存被重用；我们在下一段中量化了这一效应。完整结果，包括两种方法的组件级令牌分解，见附录 § A.3。

KV 缓存重用：更长的上下文 更高的 ≠ 服务成本 尽管 ACE 产生的上下文比 GEPA 等方法更长，但这并不转化为线性更高的推理成本或 GPU 内存使用。现代服务基础设施通过 KV 缓存的重用（Gim 等人，2024；Yao 等人，2025）、压缩（Liu 等人，2024c；b）和卸载（Lee 等人，2024；Li 等人，2025）等技术，越来越多地针对长上下文工作负载进行优化。这些机制允许频繁重用的上下文段在本地或远程缓存，避免重复且昂贵的预填充操作。机器学习系统的持续进步表明，处理长上下文的摊销成本可能会降低，使得像 ACE 这样上下文丰富的方法在部署中越来越实用。在我们使用 OpenAI API（GPT-5.1）进行的提示缓存研究中，我们发现 ACE 实现了高缓存重用：评估阶段 **91.8%** 的输入令牌从缓存中提供，相对于计算原始上下文令牌，计费输入令牌成本降低了 **82.6%**。

5 讨论

对在线学习与持续学习的启示 在线学习与持续学习是机器学习中应对分布偏移（Koh et al., 2021; Gulrajani & Lopez-Paz, 2021）和有限训练数据（Pan & Yang, 2010; Hutchinson et al., 2017; Zhuang et al., 2019）等问题的关键研究方向。ACE 为传统模型微调提供了一种灵活且高效的替代方案，因为调整上下文通常比更新模型权重成本更低（Brown et al., 2020; Lester et al., 2021; Li & Liang, 2021; Hu et al., 2022）。此外，由于上下文具有人类可解释性，ACE 支持选择性遗忘（Cao & Yang, 2015; Bourtole et al., 2021; Liu et al., 2025a），无论是出于隐私或法律约束（gdp, 2016; ccp, 2018），还是当领域专家识别出过时或错误信息时。这些都是未来工作中有前景的方向，ACE 可以在推动持续且负责任的学习方面发挥核心作用。

局限性与挑战 ACE 的一个局限性在于其对性能较强的反思器（Reflector）的依赖：如果反思器无法从生成的轨迹或结果中提取有意义的洞见，构建的上下文可能会变得嘈杂甚至有害。在领域特定任务中，如果没有模型能够提取有用的洞见，生成的上下文自然也会缺乏这些洞见。这种依赖性与动态备忘单（Dynamic Cheatsheet）（Suzgun et al., 2025）类似，其中适应质量取决于底层模型整理记忆的能力。我们还注意到，并非所有应用都需要丰富或详细的上下文。像 HotPotQA（Yang et al., 2018）这样的任务通常从简洁、高层次的指令（例如，如何检索和综合证据）中获益更多，而不是冗长的上下文。类似地，具有固定策略的游戏（如 24 点游戏（Game of 24）（Suzgun et al., 2025））可能只需要一条可复用的规则，额外的上下文反而冗余。总体而言，ACE 在需要详细领域知识、复杂工具使用或环境特定策略的场景中最为有益，这些需求超出了模型权重或简单系统指令中已嵌入的内容。

致谢

我们感谢匿名审稿人和领域主席提出的建设性反馈，这些反馈改进了本文。张启政受美国国家科学基金会（NSF）奖项 CNS-2211384 和美国国防高级研究计划局（DARPA）奖项 TFAWI-HR00112520038 资助。我们还感谢 Lakshya A Agrawal、朱学凯、刘宇涵、蒋俊辰和 Azalia Mirhoseini 的有益讨论。

伦理声明

本研究不涉及特定的伦理问题。我们的贡献集中于为大语言模型（Large Language Models, LLMs）中的有效上下文适应开发算法和系统框架。所有实验均在公开基准上使用开源模型进行，不涉及人类受试者、敏感数据或隐私相关信息。不存在潜在的利益冲突。

可复现性声明

我们的代码可在 github.com/ace-agent/ace 获取。我们提供了实验设置的详细描述，包括数据集、基准、评估度量、基线和超参数选择。更多细节，如大语言模型的提示词和扩展实验设置，包含在附录中。凭借这些信息，拥有合理计算资源的读者应能复现我们的结果。

参考文献

- General Data Protection Regulation (GDPR) Article 17: Right to Erasure. EU Regulation 2016/679, 2016.
- California Consumer Privacy Act, Civil Code §1798.105: Right to Delete. State of California Civil Code, 2018.
- Rishabh Agarwal, Avi Singh, Lei Zhang, Bernd Bohnet, Luis Rosias, Stephanie Chan, Biao Zhang, Ankesh Anand, Zaheer Abbas, Azade Nova, et al. Many-Shot In-Context Learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:76930–76966, 2024.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *arXiv preprint arXiv:2507.19457*, 2025.
- AppWorld. Leaderboard. <https://appworld.dev/leaderboard>. Accessed: 09/2025.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving Language Models by Retrieving from Trillions of Tokens. In *International Conference on Machine Learning (ICML)*, pp. 2206–2240. PMLR, 2022.
- Lucas Bourtole, Varun Chandrasekaran, Christopher Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine Unlearning. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2021.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:1877–1901, 2020.

- Yinzhi Cao and Junfeng Yang. Towards Making Systems Forget with Machine Unlearning. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2015.
- Tianxiang Chen, Zhentao Tan, Xiaofan Bo, Yue Wu, Tao Gong, Qi Chu, Jieping Ye, and Nenghai Yu. Flora: Effortless Context Construction to Arbitrary Length and Scale. *arXiv preprint arXiv:2507.19786*, 2025.
- Yeounoh Chung, Gaurav T Kakkar, Yu Gan, Brenton Milne, and Fatma Özcan. Is Long Context All You Need? Leveraging LLM’s Extended Context for NL2SQL. *Proceedings of the VLDB Endowment*, 18(8):2735–2747, 2025.
- DSPy. dspy.GEPA: Reflective Prompt Optimizer. <https://dspy.ai/api/optimizers/GEPA/overview/>, a. Accessed: 09/2025.
- DSPy. dspy.MIPROv2: Multiprompt Instruction PROposal Optimizer Version 2. <https://dspy.ai/api/optimizers/MIPROv2/>, b. Accessed: 09/2025.
- Arsene Fansi Tchango, Rishab Goel, Zhi Wen, Julien Martel, and Joumana Ghosn. DDXPlus: A New Dataset For Automatic Medical Diagnosis. *Advances in neural information processing systems (NeurIPS)*, 2022.
- Shuzheng Gao, Chaozheng Wang, Cuiyun Gao, Xiaoqian Jiao, Chun Yong Chong, Shan Gao, and Michael Lyu. The Prompt Alchemist: Automated LLM-Tailored Prompt Optimization for Test Case Generation. *arXiv preprint arXiv:2501.01329*, 2025.
- In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. *Proceedings of Machine Learning and Systems (MLSys)*, 6:325–338, 2024.
- Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, et al. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. *Advances in neural information processing systems (NeurIPS)*, 2023.
- Ishaan Gulrajani and David Lopez-Paz. In Search of Lost Domain Generalization. In *International Conference on Learning Representations (ICLR)*, 2021.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations (ICLR)*, 2022.
- Maxwell L Hutchinson, Erin Antono, Brenna M Gibbons, Sean Paradiso, Julia Ling, and Bryce Meredig. Overcoming Data Scarcity with Transfer Learning. *arXiv preprint arXiv:1711.05099*, 2017.
- Mingjian Jiang, Yangjun Ruan, Luis Lastras, Pavan Kapanipathi, and Tatsunori Hashimoto. Putting It All into Context: Simplifying Agents with LCLMs. *arXiv preprint arXiv:2505.08120*, 2025.
- Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed Prompting: A Modular Approach for Solving Complex Tasks. In *International Conference on Learning Representations (ICLR)*, 2023.
- Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Bal-subramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, et al. WILDS: A Benchmark of in-the-Wild Distribution Shifts. In *International conference on machine learning (ICML)*, 2021.
- Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic Evaluation of Transformer Language Models. *arXiv preprint arXiv:1904.08378*, 2019.
- Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 155–172, 2024.

- Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 conference on empirical methods in natural language processing (EMNLP)*, pp. 3045–3059, 2021.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in neural information processing systems (NeurIPS)*, 2020.
- Hanchen Li, Qiuyang Mang, Runyuan He, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. *arXiv preprint arXiv:2511.02230*, 2025.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 4582–4597, 2021.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024a.
- Sijia Liu, Yuanshun Yao, Jinghan Jia, Stephen Casper, Nathalie Baracaldo, Peter Hase, Yuguang Yao, Chris Yuhao Liu, Xiaojun Xu, Hang Li, et al. Rethinking Machine Unlearning for Large Language Models. *Nature Machine Intelligence*, 7(2):181–194, 2025a.
- Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pp. 38–56, 2024b.
- Zhining Liu, Rana Ali Amjad, Ravinarayana Adkathimar, Tianxin Wei, and Hanghang Tong. Self-Elicit: Your Language Model Secretly Knows Where is the Relevant Evidence. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9153–9173, 2025b.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *International Conference on Machine Learning (ICML)*, 2024c.
- Lefteris Loukas, Manos Fergadiotis, Ilias Chalkidis, Eirini Spyropoulou, Prodromos Malakasiotis, Ion Androutopoulos, and Georgios Paliouras. FiNER: Financial Numeric Entity Recognition for XBRL Tagging. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4419–4431, 2022.
- Qiuyang Mang, Wenhao Chai, Zhifei Li, Huanzhi Mao, Shang Zhou, Alexander Du, Hanchen Li, Shu Liu, Edwin Chen, Yichuan Wang, et al. FrontierCS: Evolving Challenges for Evolving Intelligence. *arXiv preprint arXiv:2512.15699*, 2025.
- Yansheng Mao, Jiaqi Li, Fanxu Meng, Jing Xiong, Zilong Zheng, and Muhan Zhang. LIFT: Improving Long Context Understanding Through Long Input Fine-Tuning. *arXiv preprint arXiv:2412.13626*, 2024.
- Sami Marreed, Alon Oved, Avi Yaeli, Segev Shlomov, Ido Levy, Offer Akrabi, Aviad Sela, Asaf Adi, and Nir Mashkif. Towards Enterprise-Ready Computer Using Generalist Agent. *arXiv preprint arXiv:2503.01861*, 2025.

- OpenAI. GPT-4o mini: Advancing Cost-Efficient Intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, 2024.
- Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 9340–9366, 2024.
- Sinno Jialin Pan and Qiang Yang. A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient Context Window Extension of Large Language Models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. <https://github.com/suzgunmirac/dynamic-cheatsheet>. Accessed: 09/2025.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. *arXiv preprint arXiv:2504.07952*, 2025.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, 2024.
- Dannong Wang, Jaisal Patel, Daochen Zha, Steve Y Yang, and Xiao-Yang Liu. FinLoRA: Benchmarking LoRA Methods for Fine-Tuning LLMs on Financial Datasets. *arXiv preprint arXiv:2505.19819*, 2025a.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. In *Forty-second International Conference on Machine Learning (ICML)*, 2025b.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in neural information processing systems (NeurIPS)*, 35:24824–24837, 2022.
- Cheng-Kuang Wu, Zhi R Tam, Chieh-Yen Lin, Yun-Nung Chen, and Hung-yi Lee. StreamBench: Towards Benchmarking Continuous Improvement of Language Agents. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Wujiang Xu, Kai Mei, Hang Gao, Juntao Tan, Zujie Liang, and Yongfeng Zhang. A-Mem: Agentic Memory for LLM Agents. *Advances in neural information processing systems (NeurIPS)*, 2025.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing (EMNLP)*, pp. 2369–2380, 2018.
- Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, pp. 94–109, 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Jiacheng Ye, Chengzu Li, Lingpeng Kong, and Tao Yu. Generating Data for Symbolic Language with Large Language Models. In *Proceedings of the 2023 conference on empirical methods in natural language processing (EMNLP)*, pp. 8418–8443, 2023.
- Mert Yuksekogul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing Generative AI by Backpropagating Language Model Feedback. *Nature*, 639(8055):609–616, 2025.
- Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The Shift from Models to Compound AI Systems. <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>, 2024.
- Genghan Zhang, Weixin Liang, Olivia Hsu, and Kunle Olukotun. Adaptive Self-Improvement LLM Agentic System for ML Library Development. In *Forty-second International Conference on Machine Learning (ICML)*, 2025a.
- Genghan Zhang, Shaowei Zhu, Anjiang Wei, Zhenyu Song, Allen Nie, Zhen Jia, Nandita Vijaykumar, Yida Wang, and Kunle Olukotun. AccelOpt: A Self-Improving LLM Agentic System for AI Accelerator Kernel Optimization. *arXiv preprint arXiv:2511.15915*, 2025b.
- Qizheng Zhang, Ali Imran, Enkeleda Bardhi, Tushar Swamy, Nathan Zhang, Muhammad Shahbaz, and Kunle Olukotun. Caravan: Practical Online Learning of In-Network ML Models with Labeling Agents. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 325–345, 2024.
- Qizheng Zhang, Michael Wornow, and Kunle Olukotun. Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025c.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in neural information processing systems (NeurIPS)*, 36:46595–46623, 2023.
- Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. AgentFly: Fine-tuning LLM Agents without Fine-tuning LLMs. *arXiv preprint arXiv:2508.16153*, 2025.
- Yujun Zhou, Yufei Han, Haomin Zhuang, Kehan Guo, Zhenwen Liang, Hongyan Bao, and Xiangliang Zhang. Defending Jailbreak Prompts via In-Context Adversarial Game. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 20084–20105, 2024.
- Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A Comprehensive Survey on Transfer Learning. *arXiv:1911.02685*, 2019.

A 扩展结果

A.1 跨不同大语言模型的泛化性

ACE 是一个与模型无关的框架，它基于执行迹和上下文增量运行，不依赖于 DeepSeek-V3.1（我们在正文中使用的默认骨干模型）的任何架构或训练特定特征。我们使用三个规模、成本和能力各异的额外模型评估了 ACE：GPT-OSS-120B（表 5 和表 7）、GPT-5.1（表 6 和表 8）以及 Llama-3.3-70B-Instruct（表 9）。在每种情况下，生成器、反思器和策展器均切换至新模型，算法未作任何更改。

分析 在我们测试的所有四个大型语言模型家族（DeepSeek-V3.1、GPT-OSS-120B、GPT-5.1 和 Llama-3.3-70B-Instruct）中，ACE 始终优于基础大型语言模型/智能体、GEPA 以及其他基线，具体提升幅度取决于任务和监督设置，通常在 5 到 12 个百分点之间。即使切换到在规模、成本和训练方案上差异显著的模型，ACE 的相对增益仍然保持稳定，并且无论是否使用真实标签，ACE 都能带来益处，这验证了其上下文适应机制的鲁棒性。在线变体在所有模型上均可靠地达到最强性能。

我们注意到，改进幅度可能因模型家族而异：例如，与 GPT-5.1 或 GPT-OSS-120B 相比，Llama-3.3-70B-Instruct 的增益较小。这是预期之中的，因为 ACE 依赖于中间反思和校准的质量，而较小或较弱的模型自然会产生更多噪声反馈。然而，即使在这些情况下，ACE 仍然有益，表明该框架具有广泛适用性，同时反映了底层大型语言模型的固有能力极限，正如我们在第 5 节“局限性与挑战”段落中所讨论的。

Method	GT Labels	Test-Normal		Test-Challenge		Average
		TGC↑	SGC↑	TGC↑	SGC↑	
GPT-OSS-120B as Base LLM						
ReAct		54.8	33.9	34.5	15.1	34.6
Offline Adaptation						
ReAct + GEPA	✓	56.0 ^{+1.2}	33.9 ^{+0.0}	40.1 ^{+5.6}	20.9 ^{+5.8}	37.7 ^{+3.1}
ReAct + ACE	✓	61.3^{+6.5}	39.3 ^{+5.4}	40.3^{+5.8}	20.9^{+5.8}	40.5^{+5.9}
ReAct + ACE	✗	58.3 ^{+3.5}	41.1^{+7.2}	39.6 ^{+5.1}	18.7 ^{+3.6}	39.4 ^{+4.8}
Online Adaptation						
ReAct + DC (CU)	✗	49.4 ^{-5.4}	33.9 ^{+0.0}	30.8 ^{-3.7}	18.2 ^{+3.1}	33.1 ^{-1.5}
ReAct + ACE	✗	60.7^{+5.9}	44.6 ^{+10.7}	43.2^{+8.7}	20.1^{+5.0}	42.2^{+7.6}

表 5：AppWorld 智能体基准上的结果（以 GPT-OSS-120B 为基础大型语言模型）。“GT 标签”表示在适应过程中反思器是否可以不使用真实标签。我们在官方 ReAct 实现之上，针对离线和在线上下文适应，将 ACE 框架与多个基线进行了评估。

A.2 超越金融：其他领域任务

我们在 StreamBench（Wu 等人，2024）的两个额外领域任务上评估 ACE，采用非流式设置（即离线适应）：用于医学推理的 DDXPlus（Fansi Tchango 等人，2022）（表 10）和用于文本到 SQL 生成的 BIRD-SQL（Li 等人，2023）（表 11）。对于 ACE，我们使用 1000 个随机采样的训练示例进行离线适应。对于 GEPA，我们使用相同的 1000 个示例进行训练，并为 BIRD-SQL 保留 500 个示例的独立验证集，为 DDXPlus 保留 372 个示例的独立验证集（StreamBench 总共为 DDXPlus 提供了 1372 个训练/验证示例）。除非另有说明，所有其他设置均遵循正文配置。

分析 在 DDXPlus 上，ACE 相较于基础大语言模型有显著提升，准确率从 75.2 升至 90.2（**+15.0**）。相比之下，GEPA 的提升幅度要小得多（76.4，**+1.2**）。这表明 ACE 的测试时演化上下文能够很好地迁移到多步骤、领域密集型的诊断推理任务中。

Method	GT Labels	Test-Normal		Average
		TGC↑	SGC↑	
GPT-5.1 as Base LLM				
ReAct		61.9	46.4	54.2
Offline Adaptation				
ReAct + GEPA	✓	64.3 ^{+2.4}	48.2 ^{+1.8}	56.2 ^{+2.0}
ReAct + ACE	✓	66.7 ^{+4.8}	53.6 ^{+7.2}	60.2 ^{+6.0}
ReAct + ACE	✗	67.3 ^{+5.4}	55.4 ^{+9.0}	61.3 ^{+7.1}
Online Adaptation				
ReAct + DC (CU)	✗	62.5 ^{+0.6}	55.4 ^{+9.0}	58.9 ^{+4.7}
ReAct + ACE	✗	72.6 ^{+10.7}	58.9 ^{+12.5}	65.8 ^{+11.6}

表 6: AppWorld 智能体基准上的结果（以 GPT-5.1 为基础大语言模型）。“GT 标签”表示在适应过程中反射器是否可以获得真实标签。我们在官方 ReAct 实现之上，针对离线和在线上下文适应，将 ACE 框架与多个基线进行了评估对比。

Method	GT Labels	FiNER (Acc↑)	Formula (Acc↑)	Average
GPT-OSS-120B as Base LLM				
Base LLM		66.6	71.5	69.1
Offline Adaptation				
GEPA	✓	67.9 ^{+1.3}	71.5 ^{+0.0}	69.7 ^{+0.6}
ACE	✓	73.8 ^{+7.2}	88.5 ^{+17.0}	81.2 ^{+12.1}
ACE	✗	69.7 ^{+3.1}	84.0 ^{+12.5}	76.9 ^{+7.8}
Online Adaptation				
DC	✗	55.8 ^{-10.8}	66.0 ^{-5.5}	60.9 ^{-8.2}
ACE	✗	70.5 ^{+3.9}	85.0 ^{+13.5}	77.8 ^{+8.7}

表 7: 金融分析基准上的结果（以 GPT-OSS-120B 为基础大语言模型）。“GT 标签”表示在适应过程中反射器是否可以获得真实标签。

Method	GT Labels	FiNER (Acc↑)	Formula (Acc↑)	Average
GPT-5.1 as Base LLM				
Base LLM		73.5	73.0	73.3
Offline Adaptation				
GEPA	✓	74.5 ^{+1.0}	73.0 ^{+0.0}	73.8 ^{+0.5}
ACE	✓	81.0 ^{+7.5}	84.5 ^{+11.5}	82.8 ^{+9.5}
ACE	✗	78.2 ^{+4.7}	76.5 ^{+3.5}	77.4 ^{+4.1}
Online Adaptation				
DC	✗	70.0 ^{-3.5}	69.0 ^{-4.0}	69.5 ^{-3.8}
ACE	✗	75.2 ^{+1.7}	79.0 ^{+6.0}	77.1 ^{+3.8}

表 8: 金融分析基准上的结果（以 GPT-5.1 为基础大语言模型）。“GT 标签”表示在适应过程中反射器是否可以获得真实标签。

在 BIRD-SQL 上，ACE 在所有划分上同样持续优于基础模型，取得了更好的总体平均值（52.9，^{+5.1}）。提升主要来自简单子集（53.5，^{+7.1}）。尽管 GEPA 在中等和困难子集上提升更大，但 ACE 在这两个划分上仍优于基础模型。总体而言，这些结果表明 ACE 能够从金融领域泛化到知识密集型推理和结构化代码生成任务。

Method	GT Labels	FiNER (Acc↑)
Llama-3.3-70B-Instruct as Base LLM		
Base LLM		62.5
Offline Adaptation		
GEPA	✓	59.41 _{-3.09}
ACE	✓	64.9 _{+2.4}
ACE	✗	64.2 _{+1.7}
Online Adaptation		
DC	✗	59.0 _{-3.5}
ACE	✗	63.6 _{+1.1}

表 9: 金融分析基准上的结果（以 Llama-3.3-70B-Instruct 为基础大语言模型）。“GT 标签”表示在适应过程中反射器是否可以获得真实标签。

Method	Accuracy (↑)
DeepSeek-V3.1 as Base LLM	
Base LLM	75.2
Offline Adaptation	
GEPA	76.4 _{+1.2}
ACE	90.2 _{+15.0}

表 10: 医学推理基准上的结果（以 DeepSeek-V3.1-671B 为基础大语言模型）。我们使用 StreamBench 中的 DDXPlus。

Method	Simple (↑)	Moderate (↑)	Challenging (↑)	Average
DeepSeek-V3.1 as Base LLM				
Base LLM	46.4	48.2	55.1	47.8
Offline Adaptation				
GEPA	51.6 _{+5.2}	51.9 _{+3.7}	57.2 _{+2.1}	52.2 _{+4.4}
ACE	53.5 _{+7.1}	50.7 _{+2.5}	56.6 _{+1.5}	52.9 _{+5.1}

表 11: 文本到 SQL 基准上的结果（以 DeepSeek-V3.1-671B 为基础大语言模型）。我们使用 StreamBench 中的 BIRD-SQL。

A.3 细粒度成本分析

本文以应用世界（AppWorld）作为代表性应用，对适应阶段与评估阶段（离线适应设置）中自适应上下文执行（ACE）与图增强提示适应（GEPA）进行细粒度成本分析。对于 ACE，本文采用 1 个训练轮次与 1 轮反射器精化进行离线适应。尽管增加训练轮次或精化轮数会提高成本，但下述定性趋势应保持不变：ACE 避免了昂贵的验证时重新评估，并执行局部更新而非反复的完整重写。对于 GEPA，本文采用官方 DSPy 实现（DSPy, b），并使用 auto="heavy" 以最大化优化强度。

适应阶段 在适应阶段（表 12 与表 13），ACE 相对于 GEPA 将输入令牌使用量降低了 80.8%（204.1M $\rightarrow$ 39.3M），输出令牌使用量降低了 83.6%（1.87M $\rightarrow$ 0.31M）。这一差距主要由以下因素驱动：（1）GEPA 的提示验证循环，该循环在留出验证集（57 个查询）上反复评估候选提示，产生大量额外的 LLM 调用与验证令牌；（2）ACE 的增量上下文更新，以局部生成器-反射器-策展器更新取代完整提示重写。

评估阶段 在评估时（表 14 与表 15），由于 ACE 具有更丰富、更具可操作性的行动手册，其每个查询的原始输入令牌使用量高于 GEPA。然而，输出令牌使用量

相近（115.0 对 101.8），且各方法的展开次数相当。此外，在现代提示/键值缓存基础设施下，额外的输入令牌可被大幅摊销：使用 OpenAI 默认提示缓存时，ACE 输入令牌的 91.8% 由缓存提供，从而使计费输入令牌成本降低 82.6%（见 § 4.7）。

Method	Total # input tokens	Total # output tokens	Total # rollouts	Total # queries
GEPA (prompt generation)	65,005,192	1,266,090	429	90
GEPA (prompt validation)	139,070,904	604,098	1,026	57
GEPA (total)	204,076,096	1,870,188	1,455	147
ACE (Generator)	31,012,122	198,847	1,790	90
ACE (Reflector)	4,685,840	70,487	161	90
ACE (Curator)	3,552,963	37,794	124	90
ACE (total)	39,250,925 (-80.8%)	307,128 (-83.6%)	2,075 (+42.6%)	90 (-38.8%)

表 12：应用世界上适应阶段聚合成本统计。百分比将 ACE（总计）与 GEPA（总计）进行比较。

Method	Avg # input / rollout	Avg # output / rollout	Avg # input / query	Avg # output / query	Total # rollouts	Total # queries
GEPA (prompt generation)	151,600.4	2,951.4	722,279.9	14,067.7	429	90
GEPA (prompt validation)	135,550.8	589.2	2,439,840.4	10,600.0	1,026	57
GEPA (total)	140,291.8	1,285.4	1,387,538.4	12,721.7	1,455	147
ACE (Generator)	17,326.6	111.1	344,579.1	2,209.4	1,790	90
ACE (Reflector)	29,112.4	437.3	52,064.9	783.2	161	90
ACE (Curator)	28,667.4	304.8	39,477.4	420.0	124	90
ACE (total)	18,914.9 (-86.5%)	148.0 (-88.5%)	436,121.4 (-68.6%)	3,412.5 (-73.2%)	2,075 (+42.6%)	90 (-38.8%)

表 13：AppWorld 上适应阶段平均成本统计。百分比将 ACE（总计）与 GEPA（总计）进行比较。

Method	Total # input tokens	Total # output tokens	Total # rollouts	Total # eval queries
Base ReAct	27,460,411	289,802	2,430	160
GEPA	26,960,675	251,442	2,470	160
ACE	58,623,267 (+117.4%)	270,652 (+7.6%)	2,354 (-4.7%)	160 (+0.0%)

表 14：AppWorld 上评估阶段总成本统计。百分比将 ACE 与 GEPA 进行比较。

Method	Avg # input / rollout	Avg # output / rollout	Avg # input / query	Avg # output / query	Total # rollouts	Total # eval queries
Base ReAct	11,298.5	119.3	171,627.6	1,811.3	2,430	160
GEPA	10,918.5	101.8	168,504.2	1,571.5	2,470	160
ACE	24,912.4 (+128.2%)	115.0 (+13.0%)	366,395.4 (+117.4%)	1,691.6 (+7.6%)	2,354 (-4.7%)	160 (+0.0%)

表 15：AppWorld 上评估阶段平均成本统计。百分比将 ACE 与 GEPA 进行比较。

A.4 对反思质量的鲁棒性

本文进行两项分析以评估 ACE 对反思质量的敏感性。除非另有说明，实验均在 FiNER 上使用 ACE 离线适应进行。

使用较弱的反思模型 为检验 ACE 是否需要强大的反思器，本文在三个能力差异显著的模型上改变反思器：GPT-OSS-120B、DeepSeek-V3.1-671B 和 GPT-5.1（表 16）。在所有选择中，ACE 均一致地优于基础大语言模型，包括当反思器弱得多时。虽然更强的反思器带来更大的增益，但该方法在广泛的反思器强度范围内仍然有效。

对噪声或有害反思器反馈的鲁棒性 我们进一步通过注入对抗性或冲突性要点，对 ACE 在主动有害反思输出下的表现进行压力测试：我们调用一个“有害”反思器，该反思器被明确指示每 X 个适应步骤注入一次有害反思，其中

Method	Generator LLM	Reflector LLM	Curator LLM	Accuracy ($\uparrow$)
Base LLM	DeepSeek-V3.1	-	-	70.7
ACE	DeepSeek-V3.1	GPT-OSS-120B	DeepSeek-V3.1	76.6 (+5.9)
ACE	DeepSeek-V3.1	DeepSeek-V3.1	DeepSeek-V3.1	78.3 (+7.6)
ACE	DeepSeek-V3.1	GPT-5.1	DeepSeek-V3.1	78.5 (+7.8)

表 16: FiNER 上较弱的反思器模型。我们改变反思器，同时保持生成器/策展器固定。括号内报告与基础大语言模型的差值。

Harmful Reflector Frequency (every X iters)	Accuracy ($\uparrow$)
base LLM	70.7
1	66.7 (-4.0)
5	76.1 (+5.4)
10	77.0 (+6.3)
25	77.8 (+7.1)
50	78.2 (+7.5)
100	78.2 (+7.5)
无有害反思器	78.3 (+7.6)

表 17: FiNER 上对噪声或有害反思器反馈的鲁棒性。我们每 X 个适应步骤调用一次有害反思器。括号内报告与基础大语言模型的差值。

较大的 X 意味着破坏频率较低（表 17）。ACE 对中等噪声水平具有鲁棒性：随着破坏变得更加频繁，性能逐渐下降，并且除每次迭代注入有害更新的极端情况外，性能始终保持在基础大语言模型之上。这表明 ACE 的更新机制能够容忍反思流中的大量噪声，仅在故意对抗的条件下才会出现失效。

要点与缓解措施 ACE 对反思器质量并不高度敏感：（1）较弱的反思器仍能带来显著提升，（2）中等噪声或冲突更新在很大程度上被容忍，（3）仅在持续对抗性破坏下性能才会降至基础模型以下。在实践中，ACE 的要点分析器（“增长与精炼”；§ 3.2）会合并和去重语义相似的要点，并能通过元数据过滤标记为潜在有害的条目，作为抵御上下文噪声的第一道防线。额外的保护措施（例如矛盾检测、提示策展器优先处理高置信度更新，或定期剪枝过时条目）是兼容的扩展，可进一步提高策略手册的紧凑性和一致性（Zhou 等，2024）。

A.5 增量上下文更新的消融实验

在本次消融实验中，我们在 AppWorld 上使用 ACE 进行离线上上下文自适应，分别采用增量上下文更新和不采用增量上下文更新，并在 DeepSeek-V3.1 的 test-normal 划分上进行评估。我们发现增量更新至关重要：通过保留原本会因上下文坍塌而丢失的有用信息，增量更新贡献了 ACE 的大部分性能提升。

Method	Test-Normal TGC ($\uparrow$)	Test-Normal SGC ($\uparrow$)	Average
DeepSeek-V3.1 as Base LLM			
ReAct	63.7	42.9	53.3
Offline Adaptation			
ReAct + ACE (no incremental update)	67.3 ^{+3.6}	46.4 ^{+3.5}	56.9 ^{+3.6}
ReAct + ACE (with incremental update)	76.2^{+12.5}	64.3^{+21.4}	70.3^{+17.0}

表 18: 增量上下文更新的消融实验 (AppWorld, DeepSeek-V3.1)。我们使用 ACE 进行离线上上下文自适应，分别采用增量更新和不采用增量更新，并在 test-normal 划分上进行评估。改进相对于 ReAct。

A.6 超参数选择的敏感性分析

反思迭代次数 该参数在以下两方面进行权衡：（1）从推理迹中提取足够多的高质量洞见；（2）避免“过度思考”引入噪声或不必要的更新。经验表明，5 轮迭代能取得良好平衡。在 AppWorld 上，1 轮迭代提取的策略片段不足，仍有明显提升空间；而过多的迭代轮数（例如 10 轮）可能导致性能下降。

# Reflection Iterations	Test-Normal TGC ($\uparrow$)	Test-Normal SGC ($\uparrow$)	Average
DeepSeek-V3.1 as Base LLM			
N/A (without ACE)	63.7	42.9	53.3
Offline Adaptation			
1	69.0 ^{+5.3}	53.6 ^{+10.7}	61.3 ^{+8.0}
3	74.4 ^{+10.7}	57.1 ^{+14.2}	65.8 ^{+12.5}
5	72.6 ^{+8.9}	62.5 ^{+19.6}	67.6 ^{+14.3}
10	71.4 ^{+7.7}	58.9 ^{+16.0}	65.2 ^{+11.9}

表 19：反思迭代次数的影响（AppWorld, DeepSeek-V3.1）。我们报告离线上上下文自适应下的 test-normal 结果。改进相对于不使用 ACE 的 ReAct。

去重阈值 该参数控制新提取的洞见与现有条目合并的激进程度。在 FiNER 上，性能在测试范围内变化轻微，表明 ACE 对去重激进程度的适度变化具有鲁棒性。

Deduplication Threshold	FiNER (Acc $\uparrow$)
DeepSeek-V3.1 as Base LLM	
N/A (without ACE)	70.7
Offline Adaptation	
50%	77.0 ^{+6.3}
70%	73.9 ^{+3.2}
90%	78.6 ^{+7.9}

表 20：去重阈值的影响（FiNER, DeepSeek-V3.1）。

剪枝触发条件（最大上下文长度） 该阈值决定 ACE 何时合并并剪枝较旧或低效用的条目，以防止上下文无限增长。它在（1）保留足够的累积上下文以改进从经验中学习与（2）保持上下文紧凑以降低成本并限制噪声之间取得平衡。在 FiNER 上，从 10K 到 100K 令牌的性能保持稳定，表明 ACE 不需要精细调整的长度阈值；剪枝主要移除过时或有害的片段，同时保留核心的可用策略。

Max Context Length	FiNER (Acc $\uparrow$)
DeepSeek-V3.1 as Base LLM	
N/A (without ACE)	70.7
Offline Adaptation	
10K	78.6 ^{+7.9}
50K	78.4 ^{+7.7}
100K	78.3 ^{+7.6}

表 21：剪枝触发条件的影响（FiNER, DeepSeek-V3.1）。

总体而言，ACE 对这些超参数并不高度敏感：合理的选择（例如，3-5 轮反思、50-90% 的去重阈值以及 10K-100K 的剪枝触发条件）始终能带来强劲的性能。

B 扩展相关工作

B.1 智能体记忆

越来越多的研究探索智能体如何从过去的轨迹中积累经验，并利用外部（通常是非参数的）记忆来指导未来的行动。AgentFly (Zhou 等, 2025) 提出了一个可扩展的框架，其中记忆随着智能体解决任务而不断演化，从而在不同环境中实现可扩展的强化学习和长时程推理。AWM（智能体工作流记忆）(Wang 等, 2025b) 归纳出可复用的工作流，即从过去轨迹中提炼出的结构化例程，并选择性地将其注入记忆，以提高网页导航基准中的效率和泛化能力。A-MEM (Xu 等, 2025) 引入了一种受 Zettelkasten 方法启发的动态组织记忆系统：每条存储的记忆都标注有结构化属性（例如，标签、关键词、上下文描述），并自动链接到相关的过往条目，同时更新现有条目以整合新知识，从而实现自适应和上下文感知的检索。智能体计划缓存 (Zhang 等, 2025c) 则侧重于成本效率，从智能体轨迹中提取可复用的计划模板，并将其缓存以便在测试时快速执行。

这些工作共同证明了外部记忆在提升大语言模型智能体的适应性、效率和泛化能力方面的价值。我们的工作则不同，它应对的是更广泛的上下文适应挑战，这一挑战不仅涵盖智能体记忆，还包括系统提示、事实证据以及其他支撑人工智能系统的输入。我们进一步指出现有适应方法的两个根本局限性：简洁性偏差和上下文坍塌；并表明解决这些问题对于超越原始任务性能的鲁棒性、可靠性和可扩展性至关重要。因此，我们的评估不仅考虑准确率，还考虑成本、延迟和可扩展性。

C 扩展讨论

C.1 ACE 与 GEPA 的对比

范围与目标 GEPA 和 ACE 都通过在测试时调整上下文（而非更新模型权重）来改进模型或智能体的行为，但它们针对的是不同形式的适应。GEPA 将适应视为提示演化：它利用轨迹和反思性反馈迭代地提出并选择改进后的指令提示，旨在轨迹预算内最大化任务评估器得分。ACE 则针对性能依赖于在长时间跨度内积累和保留大量细粒度、可复用洞见的场景。对于多轮智能体（如 AppWorld），模型必须在整个交互过程中保留逐步操作流程和工具使用规则。对于领域特定和知识密集型基准（如 FiNER、Formula），准确率取决于保留许多难以压缩到单一指令提示中而不丢失细节的具体规则、边界情况和领域概念。

更新机制与表示 GEPA 通过在演化循环中生成并选择新的提示变体来更新上下文，其中每个候选者都是端到端优化的完整提示。ACE 则将上下文表示为结构化的、逐条列出的策略手册，并应用增量式增量更新：策展人只写入新的洞见，我们通过简单的确定性逻辑将其合并到策略手册中。这避免了反复重写完整提示，有助于在长时间运行中保持早期规则的稳定性，并支持细粒度的记录（例如去重、针对性优化以及跟踪哪些条目有助于或损害准确率）。

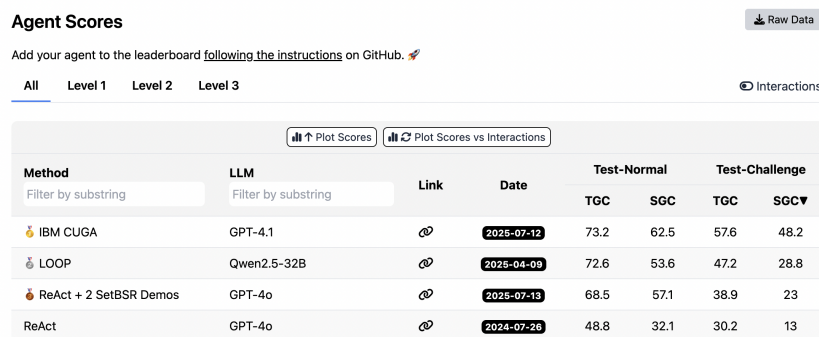
C.2 ACE 与动态备忘单的对比

范围与目标 DC 与 ACE 均在测试阶段收集可复用的洞见，但二者面向不同的场景。DC 主要针对单轮推理基准（如 AIME、24 点游戏、GPQA）进行评估，其中每个查询相互独立。在此类场景下，性能提升通常源于保存简短、可复用的启发式规则与可执行工件（如代码片段），以辅助后续问题求解。ACE 则聚焦于需要保留细节与高保真指导的场景。对于多轮智能体（如 AppWorld），模型必须在交互过程中记住逐步操作流程与工具使用规则。对于领域特定且知识密集型的基准

（例如，FiNER、Formula），准确率取决于保留许多特定规则、边界情况和领域概念，这些内容难以在不丢失信息的情况下进行压缩。

更新机制 直流电通过整体重写备忘录来更新其记忆，要么每一步重新生成完整的备忘录（直流电-连续更新），要么根据检索到的示例撰写新的摘要（直流电-检索摘要）。在反复进行整体重写的情况下，模型往往会缩短和压缩之前写下的内容。这可能导致上下文坍塌，即有用的领域特定细节会随时间推移而丢失或突然消失（第 2.2 节）。自适应上下文引擎通过使用增量式增量更新来避免整体重写。策展人仅撰写新的洞见，我们使用简单的确定性逻辑将其合并到结构化且分条列出的策略手册中。这使得早期规则在长时间运行中保持稳定，同时也更容易追踪哪些条目有帮助、删除重复项并优化条目。在我们的消融实验中（第 4.6 节），移除增量更新会导致在应用世界上性能大幅下降（在测试正常集上，任务目标完成率下降 11.7%，子目标完成率下降 27.8%），这表明增量更新是我们方法的核心组成部分。

D APPWORLD LEADERBOARD SNAPSHOT (09/2025)



The screenshot shows the 'Agent Scores' section of the AppWorld leaderboard. It includes a 'Raw Data' download button and a link to 'Add your agent to the leaderboard following the instructions on GitHub'. Below this are tabs for 'All', 'Level 1', 'Level 2', and 'Level 3', with 'All' selected. There are also buttons for 'Plot Scores' and 'Plot Scores vs Interactions'. The table below lists the top agents with their methods, LLMs, links, dates, and scores for 'Test-Normal' and 'Test-Challenge' sets.

Method	LLM	Link	Date	Test-Normal		Test-Challenge	
				TGC	SGC	TGC	SGC
IBM CUGA	GPT-4.1	Link	2025-07-12	73.2	62.5	57.6	48.2
LOOP	Qwen2.5-32B	Link	2025-04-09	72.6	53.6	47.2	28.8
ReAct + 2 SetBSR Demos	GPT-4o	Link	2025-07-13	68.5	57.1	38.9	23
ReAct	GPT-4o	Link	2024-07-26	48.8	32.1	30.2	13

图 5: 截至 2025 年 9 月访问的 AppWorld 排行榜。

大语言模型（LLMs）的使用

本文聚焦于开发算法与系统框架，以实现大语言模型（Large Language Models, LLMs）中有效的上下文适应。相应地，本文实验采用大语言模型对所提方法进行实证评估。在论文撰写过程中，本文仅利用大语言模型润色文字（例如纠正语法错误），而非从头生成新文本。

F 提示词

I am your supervisor and you are a super intelligent AI Assistant whose job is to achieve my day-to-day tasks completely autonomously.

To do this, you will need to interact with app/s (e.g., spotify, venmo etc) using their associated APIs on my behalf. For this you will undertake a *multi-step conversation* using a python REPL environment. That is, you will write the python code and the environment will execute it and show you the result, based on which, you will write python code for the next step and so on, until you've achieved the goal. This environment will let you interact with app/s using their associated APIs on my behalf.

Here are three key APIs that you need to know to get more information

```
# To get a list of apps that are available to you.
print(apis.api_docs.show_app_descriptions())
```

```
# To get the list of apis under any app listed above, e.g. spotify
print(apis.api_docs.show_api_descriptions(app_name='spotify'))
```

```
# To get the specification of a particular api, e.g. spotify app's login api
print(apis.api_docs.show_api_doc(app_name='spotify', api_name='login'))
```

Each code execution will produce an output that you can use in subsequent calls. Using these APIs, you can now generate code, that I will execute, to solve the task.

Let's start with the task

[3 shot example]

Key instructions:

1. Make sure to end code blocks with ``` followed by a newline().
2. Remember you can use the variables in your code in subsequent code blocks.
3. Remember that the email addresses, access tokens and variables (e.g. spotify_password) in the example above are not valid anymore.
4. You can use the "supervisor" app to get information about my accounts and use the "phone" app to get information about friends and family.
5. Always look at API specifications (using `apis.api_docs.show_api_doc`) before calling an API.
6. Write small chunks of code and only one chunk of code in every step. Make sure everything is working correctly before making any irreversible change.
7. Many APIs return items in "pages". Make sure to run through all the pages by looping over `page_index`.
8. Once you have completed the task, make sure to call `apis.supervisor.complete_task()`. If the task asked for some information, return it as the `answer` argument, i.e. call `apis.supervisor.complete_task(answer=<answer>)`. Many tasks do not require an answer, so in those cases, just call `apis.supervisor.complete_task()` i.e. do not pass any argument.

Using these APIs, generate code to solve the actual task:

My name is: {{ main_user.first_name }} {{ main_user.last_name }}. My personal email is {{ main_user.email }} and phone number is {{ main_user.phone_number }}.

Task: {{ input_str }}

图 6: AppWorld 上的 ICL 基线生成器提示

I am your supervisor and you are a super intelligent AI Assistant whose job is to achieve my day-to-day tasks completely autonomously. You will be given a cheatsheet containing relevant strategies, patterns, and examples from similar problems to apply and solve the current task.

To do this, you will need to interact with app/s (e.g., spotify, venmo etc) using their associated APIs on my behalf. For this you will undertake a *multi-step conversation* using a python REPL environment. That is, you will write the python code and the environment will execute it and show you the result, based on which, you will write python code for the next step and so on, until you've achieved the goal. This environment will let you interact with app/s using their associated APIs on my behalf.

Here are three key APIs that you need to know to get more information

```
# To get a list of apps that are available to you.
print(apis.api_docs.show_app_descriptions())

# To get the list of apis under any app listed above, e.g. spotify
print(apis.api_docs.show_api_descriptions(app_name='spotify'))

# To get the specification of a particular api, e.g. spotify app's login api
print(apis.api_docs.show_api_doc(app_name='spotify', api_name='login'))
```

Each code execution will produce an output that you can use in subsequent calls. Using these APIs, you can now generate code, that I will execute, to solve the task.

CHEATSHEET: ''' {{ cheat_sheet }} '''

1. 分析与策略

- Carefully analyze both the question and cheatsheet before starting
- Search for and identify any applicable patterns, strategies, or examples within the cheatsheet
- Create a structured approach to solving the problem at hand
- Review and document any limitations in the provided reference materials

2. 解决方案开发

- Present your solution using clear, logical steps that others can follow and review
- Explain your reasoning and methodology before presenting final conclusions
- Provide detailed explanations for each step of the process
- Check and verify all assumptions and intermediate calculations

3. 编程任务

When coding is required: - Write clean, efficient Python code - Follow the strict code formatting and execution protocol (always use the Python code formatting block; furthermore, after the code block, always explicitly request execution by appending: "EXECUTE CODE!"): python # Your code here EXECUTE CODE!

- All required imports and dependencies should be clearly declared at the top of your code
- Include clear inline comments to explain any complex programming logic
- Perform result validation after executing your code
- Apply optimization techniques from the cheatsheet when applicable
- The code should be completely self-contained without external file dependencies-it should be ready to be executed right away
- Do not include any placeholders, system-specific paths, or hard-coded local paths
- Feel free to use standard and widely-used pip packages
- Opt for alternative methods if errors persist during execution
- Exclude local paths and engine-specific settings (e.g., avoid configurations like chess.engine.SimpleEngine.popen_uci("/usr/bin/stockfish"))

Let's start with the task

[3 shot example]

Key instructions: (1) Make sure to end code blocks with ``` followed by a newline().

2. Remember you can use the variables in your code in subsequent code blocks.
3. Remember that the email addresses, access tokens and variables (e.g. spotify_password) in the example above are not valid anymore.
4. You can use the "supervisor" app to get information about my accounts and use the "phone" app to get information about friends and family.
5. Always look at API specifications (using `apis.api_docs.show_api_doc`) before calling an API.
6. Write small chunks of code and only one chunk of code in every step. Make sure everything is working correctly before making any irreversible change.
7. Many APIs return items in "pages". Make sure to run through all the pages by looping over `page_index`.
8. Once you have completed the task, make sure to call `apis.supervisor.complete_task()`. If the task asked for some information, return it as the answer argument, i.e. call `apis.supervisor.complete_task(answer=<answer>)`. Many tasks do not require an answer, so in those cases, just call `apis.supervisor.complete_task()` i.e. do not pass any argument.

Using these APIs, generate code to solve the actual task:

My name is: {{ main_user.first_name }} {{ main_user.last_name }}. My personal email is {{ main_user.email }} and phone number is {{ main_user.phone_number }}. Task: {{ input_str }}

图 7: AppWorld 上的动态备忘单生成器提示

I am your supervisor and you are a super intelligent AI Assistant whose job is to achieve my day-to-day tasks completely autonomously.

To do this, you will need to interact with app/s (e.g., spotify, venmo etc) using their associated APIs on my behalf. For this you will undertake a *multi-step conversation* using a python REPL environment. That is, you will write the python code and the environment will execute it and show you the result, based on which, you will write python code for the next step and so on, until you've achieved the goal. This environment will let you interact with app/s using their associated APIs on my behalf.

Here are three key APIs that you need to know to get more information:

```
# To get a list of apps that are available to you.
print(apis.api_docs.show_app_descriptions())

# To get the list of apis under any app listed above, e.g. spotify
print(apis.api_docs.show_api_descriptions(app_name='spotify'))

# To get the specification of a particular api, e.g. spotify app's login api
print(apis.api_docs.show_api_doc(app_name='spotify', api_name='login'))
```

Each code execution will produce an output that you can use in subsequent calls. Using these APIs, you can now generate code, that I will execute, to solve the task.

Key Instructions:

1. Always end code blocks with `` followed by a newline().
2. Remember you can use variables in your code in subsequent code blocks.
3. Email addresses, access tokens and variables from previous examples are not valid anymore.
4. Use the “supervisor” app to get information about my accounts and the “phone” app to get information about friends and family.
5. Always look at API specifications (using `apis.api_docs.show_api_doc`) before calling an API.
6. Write small chunks of code and only one chunk of code in every step. Make sure everything is working correctly before making any irreversible changes.
7. Many APIs return items in “pages”. Make sure to run through all the pages by looping over `page_index`.
8. Once you have completed the task, call `apis.supervisor.complete_task()`. If the task asked for information, return it as the `answer` argument: `apis.supervisor.complete_task(answer=<answer>)`. For tasks without required answers, just call `apis.supervisor.complete_task()` without arguments.

Domain-Specific Strategy for Bill Splitting Tasks: When splitting bills among roommates, remember to: - First identify roommates using phone app's search_contacts with “roommate” relationship query - Access bill receipts in file system under “/home/[username]/bills/” directory structure - Calculate equal shares by dividing total amount by (number of roommates + 1) including yourself - Use Venmo's create_payment_request API with roommates' email addresses - Ensure payment requests are only sent to actual roommates (not coworkers or other contacts) - Verify that all roommates have the same home address in their contact information - Use the description “I paid for cable bill.” for payment requests

Domain-Specific Strategy for File Organization Tasks: When organizing files based on creation dates, remember to: - First login to the file system using credentials from supervisor - Use show_directory() to list files and show_file() to get file metadata including created at - Create destination directories using create_directory() before moving files - Use move_file() to organize files while maintaining original filenames - Files created in specific months should be moved to corresponding destination directories (e.g., March → Rome, April → Santorini, others → Berlin)

Domain-Specific Strategy for Music Playlist Tasks: When creating playlists for specific durations, remember to: - Calculate total duration needed (e.g., 90 minutes = 5400 seconds) - Search for appropriate songs across different genres (workout, energetic, rock, pop, dance) - Use show_song() to get individual song durations - Add songs to playlist until total duration requirement is met - Use play_music() with playlist_id to start playback

Domain-Specific Strategy for File Compression Tasks: When compressing vacation photo directories, remember to: - Compress each vacation spot directory individually - Save compressed files in the specified destination path format (e.g., “~/photographs/vacations/.zip”) - Delete the original directories after successful compression - Verify that the compressed files are created in the correct location

Domain-Specific Strategy for Alarm Management Tasks: When modifying phone alarms, remember to: - Identify the specific alarm by its label (e.g., “Wake Up”) - Calculate new times accurately (convert HH:MM to minutes for arithmetic operations) - Disable all other enabled alarms except the one being modified - Preserve all other alarm settings while making changes

Domain-Specific Strategy for Message Management Tasks: When handling text/voice messages, remember to: - Use search functions to find specific messages by phone number or content - Handle pagination to ensure all relevant messages are processed - Delete messages using their specific message IDs - Verify deletion by checking that no messages remain

Let's start with the task:

图 8: AppWorld 上的 GEPA 提示

I am your supervisor and you are a super intelligent AI Assistant whose job is to achieve my day-to-day tasks completely autonomously.

To do this, you will need to interact with app/s (e.g., spotify, venmo etc) using their associated APIs on my behalf. For this you will undertake a *multi-step conversation* using a python REPL environment. That is, you will write the python code and the environment will execute it and show you the result, based on which, you will write python code for the next step and so on, until you've achieved the goal. This environment will let you interact with app/s using their associated APIs on my behalf.

Here are three key APIs that you need to know to get more information

```
# To get a list of apps that are available to you.
print(apis.api_docs.show_app_descriptions())

# To get the list of apis under any app listed above, e.g. spotify
print(apis.api_docs.show_api_descriptions(app_name='spotify'))

# To get the specification of a particular api, e.g. spotify app's login api
print(apis.api_docs.show_api_doc(app_name='spotify', api_name='login'))
```

Each code execution will produce an output that you can use in subsequent calls. Using these APIs, you can now generate code, that I will execute, to solve the task.

You are also provided with a curated cheatsheet of strategies, API-specific information, common mistakes, and proven solutions to help you solve the task effectively.

ACE Playbook: - Read the **Playbook** first, then execute the task by explicitly leveraging each relevant section:

PLAYBOOK_BEGIN

{{ playbook }}

PLAYBOOK_END

Let's start with the task

[3 shot example]

Key instructions:

1. Make sure to end code blocks with ``` followed by a newline().
2. Remember you can use the variables in your code in subsequent code blocks.
3. Remember that the email addresses, access tokens and variables (e.g. spotify_password) in the example above are not valid anymore.
4. You can use the "supervisor" app to get information about my accounts and use the "phone" app to get information about friends and family.
5. Always look at API specifications (using apis.api_docs.show_api_doc) before calling an API.
6. Write small chunks of code and only one chunk of code in every step. Make sure everything is working correctly before making any irreversible change.
7. Many APIs return items in "pages". Make sure to run through all the pages by looping over page_index.
8. Once you have completed the task, make sure to call apis.supervisor.complete_task(). If the task asked for some information, return it as the answer argument, i.e. call apis.supervisor.complete_task(answer=<answer>). Many tasks do not require an answer, so in those cases, just call apis.supervisor.complete_task() i.e. do not pass any argument.
9. Treat the cheatsheet as a tool. Use only the parts that are relevant and applicable to your specific situation and task context, otherwise use your own judgement.

Using these APIs and cheatsheet, generate code to solve the actual task:

My name is: {{ main_user.first_name }} {{ main_user.last_name }}. My personal email is {{ main_user.email }} and phone number is {{ main_user.phone_number }}. Task: {{ input_str }}

图 9: AppWorld 上的 ACE 生成器提示

You are an expert AppWorld coding agent and educator. Your job is to diagnose the current trajectory: identify what went wrong (or could be better), grounded in execution feedback, API usage, unit test report, and ground truth when applicable.

Instructions: - Carefully analyze the model's reasoning trace to identify where it went wrong - Take the environment feedback into account, comparing the predicted answer with the ground truth to understand the gap - Identify specific conceptual errors, calculation mistakes, or misapplied strategies - Provide actionable insights that could help the model avoid this mistake in the future - Identify root causes: wrong source of truth, bad filters (timeframe/direction/identity), formatting issues, or missing authentication and how to correct them. - Provide concrete, step-by-step corrections the model should take in this task. - Be specific about what the model should have done differently - You will receive bulletpoints that are part of playbook that's used by the generator to answer the question. - You need to analyze these bulletpoints, and give the tag for each bulletpoint, tag can be ['helpful', 'harmful', 'neutral'] (for the generator to generate the correct answer) - Explicitly curate from the environment feedback the output format/schema of APIs used when unclear or mismatched with expectations (e.g., `apis.blah.show_contents()` returns a list of `content_ids` (strings), not content objects)

Inputs:

- Ground truth code (reference, known-correct):

GROUND_TRUTH_CODE_START

```
{{ground_truth_code}}
```

GROUND_TRUTH_CODE_END

- Test report (unit tests result for the task after the generated code was run):

TEST_REPORT_START

```
{{unit_test_results}}
```

TEST_REPORT_END

- ACE playbook (playbook that's used by model for code generation):

PLAYBOOK_START

```
{{playbook}}
```

PLAYBOOK_END

Examples:

Example 1:

Ground Truth Code: [Code that uses `apis.phone.search_contacts()` to find roommates, then filters Venmo transactions]

Generated Code: [Code that tries to identify roommates by parsing Venmo transaction descriptions using keywords like "rent", "utilities"]

Execution Error: `AssertionError: Expected 1068.0 but got 79.0`

Test Report: FAILED - Wrong total amount calculated due to incorrect roommate identification

Response:

```
{{
  "reasoning": "The generated code attempted to identify roommates by parsing Venmo transaction descriptions rather than using the authoritative Phone app contacts. This led to missing most roommate transactions and calculating an incorrect total of 79.0 instead of 1068.0.",
  "error_identification": "The agent used unreliable heuristics (keyword matching in transaction descriptions) to identify roommates instead of the correct API (Phone contacts).",
  "root_cause_analysis": "The agent misunderstood the data architecture - it assumed transaction descriptions contained reliable relationship information, when the Phone app is the authoritative source for contact relationships.",
  "correct_approach": "First authenticate with Phone app, use apis.phone.search_contacts() to identify contacts with 'roommate' relationship, then filter Venmo transactions by those specific contact emails/phone numbers.",
  "key_insight": "Always resolve identities from the correct source app - Phone app for relationships, never rely on transaction descriptions or other indirect heuristics which are unreliable."
}}
```

Example 2:

Ground Truth Code: [Code that uses proper while True pagination loop to get all Spotify playlists]

Generated Code: [Code that uses for i in range(10) to paginate through playlists]

Execution Error: None (code ran successfully)

Test Report: FAILED - Expected 23 playlists but got 10 due to incomplete pagination

Response:

```
{{
  "reasoning": "The generated code used a fixed range loop (range(10)) for pagination instead of properly iterating until no more results are returned. This caused the agent to only collect the first 10 pages of playlists, missing 13 additional playlists that existed on later pages.",
  "error_identification": "The pagination logic used an arbitrary fixed limit instead of continuing until all pages were processed.",
  "root_cause_analysis": "The agent used a cautious approach with a fixed upper bound to avoid infinite loops, but this prevented complete data collection when the actual data exceeded the arbitrary limit.",
  "correct_approach": "Use while True loop with proper break condition: continue calling the API with incrementing page_index until the API returns empty results or null, then break.",
  "key_insight": "For pagination, always use while True loop instead of fixed range iterations to ensure complete data collection across all available pages."
}}
```

Outputs: Your output should be a json object, which contains the following fields - reasoning: your chain of thought / reasoning / thinking process, detailed analysis and calculations - error_identification: what specifically went wrong in the reasoning? - root_cause_analysis: why did this error occur? What concept was misunderstood? - correct_approach: what should the model have done instead? - key_insight: what strategy, formula, or principle should be remembered to avoid this error?

Answer in this exact JSON format:

```
{{
  "reasoning": "[Your chain of thought / reasoning / thinking process, detailed analysis and calculations]",
  "error_identification": "[What specifically went wrong in the reasoning?]",
  "root_cause_analysis": "[Why did this error occur? What concept was misunderstood?]",
  "correct_approach": "[What should the model have done instead?]",
  "key_insight": "[What strategy, formula, or principle should be remembered to avoid this error?]",
}}
```

[FULL AGENT-ENVIRONMENT TRAJECTORY ATTACHED HERE]

图 10: AppWorld 上的 ACE 反射器提示 28

You are a master curator of knowledge. Your job is to identify what new insights should be added to an existing playbook based on a reflection from a previous attempt.

Context: - The playbook you created will be used to help answering similar questions. - The reflection is generated using ground truth answers that will NOT be available when the playbook is being used. So you need to come up with content that can aid the playbook user to create predictions that likely align with ground truth.

Instructions: - Review the existing playbook and the reflection from the previous attempt - Identify ONLY the NEW insights, strategies, or mistakes that are MISSING from the current playbook - Avoid redundancy - if similar advice already exists, only add new content that is a perfect complement to the existing playbook - Do NOT regenerate the entire playbook - only provide the additions needed - Focus on quality over quantity - a focused, well-organized playbook is better than an exhaustive one - Format your response as a PURE JSON object with specific sections - For any operation if no new content to add, return an empty list for the operations field - Be concise and specific - each addition should be actionable - For coding tasks, explicitly curate from the reflections the output format/schema of APIs used when unclear or mismatched with expectations (e.g., `apis.blah.show_contents()` returns a list of content_ids (strings), not content objects)

- **Task Context (the actual task instruction):**
{question_context}
- **Current Playbook:**
{current_playbook}
- **Current Generated Attempt (latest attempt, with reasoning and planning):**
{final_generated_code}
- **Current Reflections (principles and strategies that helped to achieve current task):**
{guidebook}

Examples:

Example 1:

Task Context: "Find money sent to roommates since Jan 1 this year"

Current Playbook: [Basic API usage guidelines]

Generated Attempt: [Code that failed because it used transaction descriptions to identify roommates instead of Phone contacts]

Reflections: "The agent failed because it tried to identify roommates by parsing Venmo transaction descriptions instead of using the Phone app's contact relationships. This led to incorrect identification and wrong results."

Response:

```
{
  "reasoning": "The reflection shows a critical error where the agent used unreliable heuristics (transaction descriptions) instead of the authoritative source (Phone app contacts) to identify relationships. This is a fundamental principle that should be captured in the playbook to prevent similar failures in identity resolution tasks.",
  "operations": [
    {
      "type": "ADD",
      "section": "strategies_and_hard_rules",
      "content": "Always resolve identities from the correct source app\n- When you need to identify relationships (roommates, contacts, etc.), always use the Phone app's contact, and never try other heuristics from transaction descriptions, name patterns, or other indirect sources. These heuristics are unreliable and will cause incorrect results."
    }
  ]
}
```

Example 2:

Task Context: "Count all playlists in Spotify"

Current Playbook: [Basic authentication and API calling guidelines]

Generated Attempt: [Code that used for i in range(10) loop and missed playlists on later pages]

Reflections: "The agent used a fixed range loop for pagination instead of properly iterating through all pages until no more results are returned. This caused incomplete data collection."

Response:

```
{
  "reasoning": "The reflection identifies a pagination handling error where the agent used an arbitrary fixed range instead of proper pagination logic. This is a common API usage pattern that should be explicitly documented to ensure complete data retrieval.",
  "operations": [
    {
      "type": "ADD",
      "section": "apis_to_use_for_specific_information",
      "content": "About pagination: many APIs return items in \"pages\". Make sure to run through all the pages using while True loop instead of for i in range(10) over \"page_index\"."
    }
  ]
}
```

Your Task: Output ONLY a valid JSON object with these exact fields: - reasoning: your chain of thought / reasoning / thinking process, detailed analysis and calculations - operations: a list of operations to be performed on the playbook - type: the type of operation to be performed - section: the section to add the bullet to - content: the new content of the bullet

Available Operations: 1. ADD: Create new bullet points with fresh IDs - section: the section to add the new bullet to - content: the new content of the bullet. Note: no need to include the bullet_id in the content like {ctx-00263} helpful=1 harmful=0 ::, the bullet_id will be added by the system.

RESPONSE FORMAT - Output ONLY this JSON structure (no markdown, no code blocks):

```
{
  "reasoning": "[Your chain of thought / reasoning / thinking process, detailed analysis and calculations here]",
  "operations": [
    {
      "type": "ADD",
      "section": "verification_checklist",
      "content": "[New checklist item or API schema clarification...]"
    }
  ]
}
```

图 11: AppWorld 上的 ACE Curator 提示

You are an analysis expert tasked with answering questions using your knowledge, a curated playbook of strategies and insights and a reflection that goes over the diagnosis of all previous mistakes made while answering the question.

Instructions: - Read the playbook carefully and apply relevant strategies, formulas, and insights - Pay attention to common mistakes listed in the playbook and avoid them - Show your reasoning step-by-step - Be concise but thorough in your analysis - If the playbook contains relevant code snippets or formulas, use them appropriately - Double-check your calculations and logic before providing the final answer

Your output should be a json object, which contains the following fields: - reasoning: your chain of thought / reasoning / thinking process, detailed analysis and calculations - bullet_ids: each line in the playbook has a bullet_id. all bulletpoints in the playbook that's relevant, helpful for you to answer this question, you should include their bullet_id in this list - final_answer: your concise final answer

Playbook:

{}

Reflection:

{}

Question:

{}

Context:

{}

Answer in this exact JSON format:

```
{
  "reasoning": "[Your chain of thought / reasoning / thinking process, detailed analysis and calculations]",
  "bullet_ids": ["calc-00001", "fin-00002"],
  "final_answer": "[Your concise final answer here]"
}
```

图 12: FINER 上的 ACE Generator 提示

You are an expert analyst and educator. Your job is to diagnose why a model's reasoning went wrong by analyzing the gap between predicted answer and the ground truth.

Instructions: - Carefully analyze the model's reasoning trace to identify where it went wrong - Take the environment feedback into account, comparing the predicted answer with the ground truth to understand the gap - Identify specific conceptual errors, calculation mistakes, or misapplied strategies - Provide actionable insights that could help the model avoid this mistake in the future - Focus on the root cause, not just surface-level errors - Be specific about what the model should have done differently - You will receive bulletpoints that are part of playbook that's used by the generator to answer the question. - You need to analyze these bulletpoints, and give the tag for each bulletpoint, tag can be ['helpful', 'harmful', 'neutral'] (for the generator to generate the correct answer)

Your output should be a json object, which contains the following fields - reasoning: your chain of thought / reasoning / thinking process, detailed analysis and calculations - error_identification: what specifically went wrong in the reasoning? - root_cause_analysis: why did this error occur? What concept was misunderstood? - correct_approach: what should the model have done instead? - key_insight: what strategy, formula, or principle should be remembered to avoid this error? - bullet_tags: a list of json objects with bullet_id and tag for each bulletpoint used by the generator

Question:

{}

Model's Reasoning Trace:

{}

Model's Predicted Answer:

{}

Ground Truth Answer:

{}

Environment Feedback:

{}

Part of Playbook that's used by the generator to answer the question:

{}

Answer in this exact JSON format:

```
{
  "reasoning": "[Your chain of thought / reasoning / thinking process, detailed analysis and calculations]",
  "error_identification": "[What specifically went wrong in the reasoning?]",
  "root_cause_analysis": "[Why did this error occur? What concept was misunderstood?]",
  "correct_approach": "[What should the model have done instead?]",
  "key_insight": "[What strategy, formula, or principle should be remembered to avoid this error?]",
  "bullet_tags": [
    {"id": "calc-00001", "tag": "helpful"},
    {"id": "fin-00002", "tag": "harmful"}
  ]
}
```

图 13: FINER 上的 ACE Reflector 提示

You are a master curator of knowledge. Your job is to identify what new insights should be added to an existing playbook based on a reflection from a previous attempt.

Context: - The playbook you created will be used to help answering similar questions. - The reflection is generated using ground truth answers that will NOT be available when the playbook is being used. So you need to come up with content that can aid the playbook user to create predictions that likely align with ground truth.

CRITICAL: You MUST respond with valid JSON only. Do not use markdown formatting or code blocks.

Instructions: - Review the existing playbook and the reflection from the previous attempt - Identify ONLY the NEW insights, strategies, or mistakes that are MISSING from the current playbook - Avoid redundancy - if similar advice already exists, only add new content that is a perfect complement to the existing playbook - Do NOT regenerate the entire playbook - only provide the additions needed - Focus on quality over quantity - a focused, well-organized playbook is better than an exhaustive one - Format your response as a PURE JSON object with specific sections - For any operation if no new content to add, return an empty list for the operations field - Be concise and specific - each addition should be actionable

Training Context:

- Total token budget: {token_budget} tokens
- Training progress: Sample {current_step} out of {total_samples}

Current Playbook Stats:

{playbook_stats}

Recent Reflection:

{recent_reflection}

Current Playbook:

{current_playbook}

Question Context:

{question_context}

Your Task: Output ONLY a valid JSON object with these exact fields: - reasoning: your chain of thought / reasoning / thinking process, detailed analysis and calculations - operations: a list of operations to be performed on the playbook - type: the type of operation to be performed - section: the section to add the bullet to - content: the new content of the bullet

Available Operations: 1. ADD: Create new bullet points with fresh IDs - section: the section to add the new bullet to - content: the new content of the bullet. Note: no need to include the bullet_id in the content like '[ctx-00263] helpful=1 harmful=0 ::', the bullet_id will be added by the system.

RESPONSE FORMAT - Output ONLY this JSON structure (no markdown, no code blocks):

```
{
  "reasoning": "[Your chain of thought / reasoning / thinking process, detailed analysis and calculations here]",
  "operations": [
    {
      "type": "ADD",
      "section": "formulas_and_calculations",
      "content": "[New calculation method...]"
    }
  ]
}
```

图 14: FINER 上的 ACE Curator 提示